ProcVLM: Learning Procedure-Grounded Progress Rewards for Robotic Manipulation

Youhe Feng[1,2*] **Hansen Shi**[1] **Haoyang Li**[1,2] **Xinlei Guo**[1] **Yang Wang**[1]
Chengyang Zhang[1] **Jinkai Zhang**[1] **Xiaohan Zhang**[5] **Jie Tang**[4,5] **Jing Zhang**[1,3†]
[1]School of Information, Renmin University of China, Beijing, China
[2]Key Laboratory of Data Engineering and Knowledge Engineering, Beijing, China
[3]Engineering Research Center of Database and Business Intelligence, Beijing, China
[4]Department of Computer Science and Technology, Tsinghua University, Beijing, China
[5]Zhipu AI

Abstract

Long-horizon robotic manipulation requires dense feedback that reflects how a task advances through its procedural stages, not merely whether the final outcome is successful. Existing reward models often rely on trajectory-level success labels or time-based interpolation, which can conflate elapsed time with true task progress and therefore fail to capture unfinished steps, stagnation, and failure states. We present ProcVLM, a progress-aware vision-language model that learns procedure-grounded progress as a dense reward signal for manipulation. Rather than deriving progress from terminal outcomes or temporal proxies, ProcVLM grounds progress estimation in procedural structure and intra-stage visual change, and further adopts a reasoning-before-estimation paradigm that infers the remaining atomic actions before estimating task progress. Specifically, we construct this supervision by synthesizing frame-level subtask-semantic annotations, assigning progress budgets according to subtask structure, and distributing each budget based on intra-subtask visual change. To train ProcVLM at scale, we build a standardized procedural supervision synthesis pipeline and construct ProcCorpus-60M from 30 embodied datasets with 60M annotated frames, from which we derive ProcVQA for procedure-aware pretraining, with progress estimation as the central task alongside action segmentation and future planning. Experiments on ProcVQA and reward-model benchmarks show that ProcVLM improves embodied procedural reasoning and yields more discriminative trajectory-internal progress estimates than representative baselines, supporting its use as a dense reward model for downstream reward-guided policy optimization.

1 Introduction

Recent vision-language-action (VLA) models have substantially improved policy generalization from large-scale robot demonstration [6–8, 18, 19, 32, 35, 45–49, 65]. However, demonstration-centric pretraining is constrained by the cost of collecting diverse robot data and the difficulty of adapting pretrained policies to new tasks, environments, and failure modes. This has motivated reward-guided policy improvement paradigms, where VLA policies improve beyond offline demonstrations through autonomous experience, expert corrections, learned reward feedback, or self-improvement signals [2, 17, 63, 70]. Such paradigms require reward models that provide dense, task-conditioned feedback on task progress, remaining steps, stagnation, and failure.

*Work done during the internship at Zhipu AI.
†Corresponding author. zhang-jing@ruc.edu.cn

Preprint.

A central challenge is that existing robot datasets rarely provide dense supervision for procedural progress. Most large-scale corpora contain task instructions, observations, actions, and sometimes trajectory-level success labels, but lack explicit annotations of procedural structure, such as subtask boundaries and remaining actions [31, 47, 59]. As a result, robotic reward or progress models often resort to indirect supervision from sparse outcomes, time-based interpolation, temporal-difference learning, or comparison- and preference-based signals [22, 34, 37, 58, 70, 73]. However, time is not progress in long-horizon manipulation. These tasks unfold through multiple semantic stages with uneven progress rates: a policy may spend many steps retrying a grasp, remain stuck in a local stage, or recover from an earlier failure. In such cases, a later frame is not necessarily closer to successful completion, and the same robot action can correspond to different progress changes depending on the current subtask. Therefore, transferable dense rewards should be grounded in procedural structure rather than treated as a simple function of timestep.

Motivated by this observation, we introduce **ProcVLM**, a progress-aware embodied VLM that predicts dense progress rewards grounded in task procedures. ProcVLM grounds progress in procedural structure rather than timestep: a valid progress label should (i) respect subtask semantics, including boundaries, completed steps, and remaining actions, and (ii) capture within-stage advancement through intra-subtask perceptual change. On top of this supervision, ProcVLM follows a reasoning-before-estimation paradigm—it first infers the current execution stage and remaining atomic actions, and then predicts a continuous progress score conditioned on explicit procedural reasoning. This coupling makes reward prediction more sensitive to stage transitions, unfinished steps, stagnation, and failure states.

To train ProcVLM at scale, we develop a highly efficient and scalable procedural supervision synthesis pipeline for embodied trajectory annotation. The pipeline decomposes raw manipulation trajectories into procedural subtasks and annotates frame-level cues such as state reasoning, completion status, and remaining actions. Using this pipeline, we construct **ProcCorpus-60M** from 30 embodied manipulation datasets, covering 400K trajectories and 60M annotated frames. We further convert these annotations into **ProcVQA**, a 20B-token VLM training corpus centered on task progress estimation and complemented by action segmentation and future planning as auxiliary process-reasoning tasks. ProcVLM is trained in two stages: large-scale procedure-aware pretraining on ProcVQA to learn general procedural representations, followed by refinement on a curated subset to sharpen subtask alignment and progress estimation. Empirically, ProcVLM improves procedural understanding on ProcVQA, produces more discriminative trajectory-internal progress estimates than representative reward-model baselines, and supports sample-efficient one-shot adaptation on RoboFAC [66]. Moreover, ProcVLM-guided reward fine-tuning stabilizes VLA policy optimization on noisy real-robot demonstrations, yielding higher early-stage success rates than supervised fine-tuning in our experiments. Our contributions are:

- We introduce *ProcVLM*, a progress-aware embodied VLM that predicts dense procedure-grounded progress rewards through reasoning-before-estimation, linking continuous progress estimation to subtask semantics, remaining actions, and within-stage advancement.
- We develop a scalable procedural supervision synthesis pipeline for converting raw manipulation trajectories into frame-level subtask-structured annotations. It yields *ProcCorpus-60M*, *ProcVQA*, and a two-stage ProcVLM training pipeline with large-scale pretraining followed by curated refinement.
- We systematically evaluate ProcVLM across procedure-aware understanding, reward modeling, cross-task adaptation, and downstream policy optimization. The results show that procedure-grounded progress estimation enables transferable reward modeling and supports stable policy improvement.

2 Related Work

Embodied VLMs for Procedural Reasoning. A parallel line of work studies how VLMs can support procedural understanding in robotics beyond direct action prediction. Early language-grounded robotics systems leverage large foundation models for high-level planning, affordance grounding, program synthesis, and spatial-constraint reasoning [1, 23, 24, 38, 56]. Models such as PaLM-E, VIMA, and Gato further extend this idea by integrating language, vision, and embodied observations into general-purpose architectures for planning and interactive decision making [14, 29, 50]. Robotics-

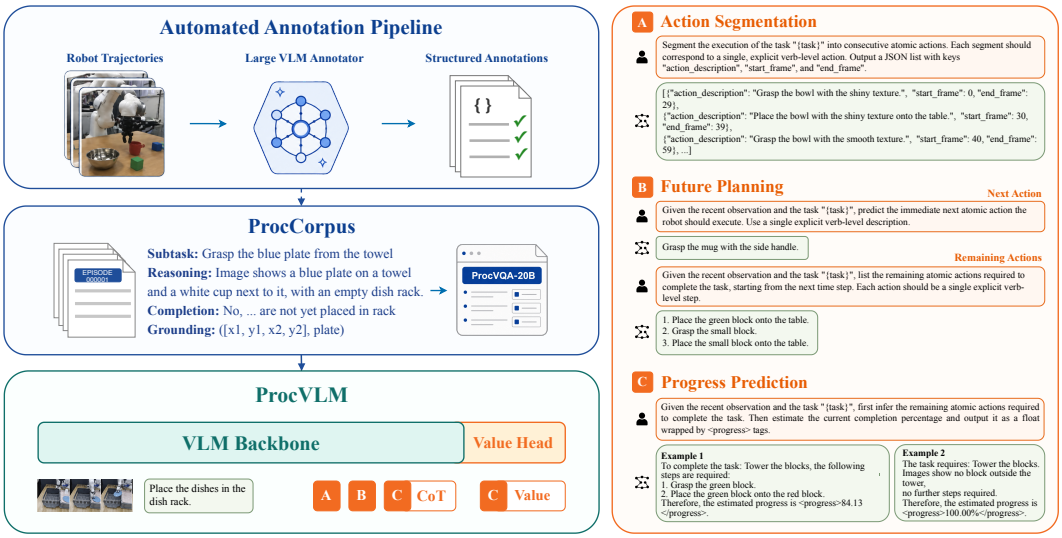


Figure 1: Overview of ProcVLM. We first synthesize frame-wise procedural annotations from robot trajectories using a large VLM annotator, forming ProcCorpus-60M. These annotations are converted into ProcVQA, which contains three procedure-aware VQA task families shown in the figure: action segmentation, future planning, and task progress prediction. ProcVLM is trained on ProcVQA to learn procedural understanding and can further provide progress-based reward signals for downstream reward-guided policy optimization.

oriented VQA, progress-reasoning, and embodied reasoning datasets further push this direction toward grounded long-horizon understanding, with RoboVQA introducing large-scale robotics-focused video-text supervision, RoboBrain unifying planning, affordance perception, and trajectory prediction in a robotic foundation model, and PROGRESSLM evaluating task-progress reasoning in VLMs [28, 54, 72]. Recent embodied reasoning models further extend VLMs toward spatial grounding, keypoint reasoning, task decomposition, and reinforced embodied reasoning, as represented by ReKep, Gemini Robotics-ER, and related systems [18, 19, 21, 25, 40, 51, 68].

Reward Modeling for Manipulation. Reward design remains a central bottleneck for applying RL to robotic manipulation. Earlier robot RL methods have addressed sparse reward supervision through goal relabeling, goal-conditioned value learning, offline value learning, and more recent action-chunked optimization for long-horizon manipulation and VLA fine-tuning [3, 9, 22, 36]. Recent foundation-model-based reward methods use pretrained semantic and visual priors for success detection, task-conditioned feedback, and reward generation, as shown by ReWiND, RoboCLIP, RoboReward, and related systems [15, 26, 30, 34, 43, 57, 71]. More specifically, VLM-based reward and progress models seek denser feedback beyond sparse success labels. TOPReward probes video-VLM likelihoods to estimate task progress, Robometer scales Qwen3VL-based video-language reward modeling by combining frame-level progress/success prediction with trajectory-comparison preference learning, and RoboDopamine learns step-aware process rewards from multi-view inputs through step-wise reward discretization and multi-perspective reward fusion [10, 37, 58]. Related dense-reward frameworks further use large VLMs, LLMs, VLA critics, or policy-internal signals to support online refinement and long-horizon policy improvement [41, 44, 62, 67, 70, 73]. While these methods provide increasingly dense feedback from visual-state changes, success calibration, preference learning, or policy rollouts, their progress labels often rely on whole-trajectory completion interpolation with limited explicit procedural constraints.

3 Methods

3.1 Procedural Supervision Synthesis

Large-scale embodied manipulation datasets contain diverse task executions, but their supervision is usually limited to task instructions, action trajectories, or coarse episode-level outcomes [31, 47, 59].

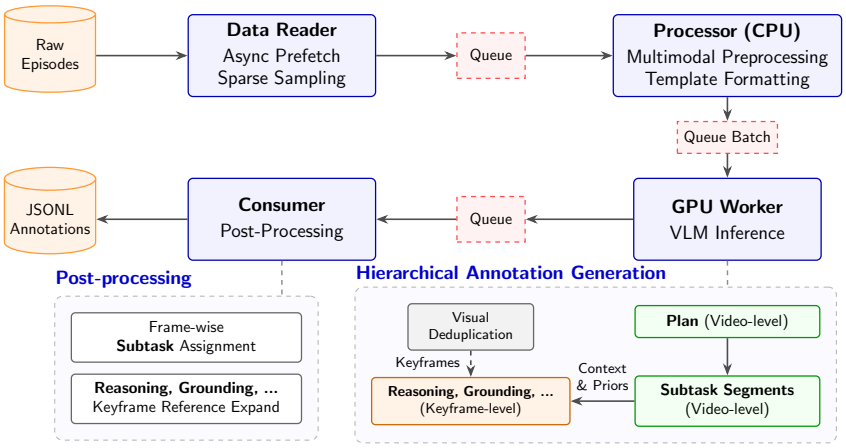


Figure 2: Overview of the procedural supervision synthesis pipeline. Raw episodes are processed through asynchronous data reading, multimodal preprocessing, VLM-based hierarchical annotation generation, and post-processing to produce JSONL annotations with frame-wise subtask labels and procedural reasoning.

To learn procedure-grounded progress rewards, we develop a scalable procedural supervision synthesis pipeline that uses large vision-language models as automatic annotators [4, 61] to convert raw trajectories into frame-wise annotations of subtask stages, completion states, and remaining actions, forming *ProcCorpus-60M*.

3.1.1 Hierarchical Annotation Pipeline

Figure 2 illustrates the annotation pipeline. To improve throughput, we decouple data reading, multimodal preprocessing, VLM inference, and post-processing into queue-connected workers. This design overlaps CPU-side input construction with GPU-side inference, reducing stalls during large-scale trajectory annotation. The resulting pipeline processes up to 4M keyframes per day on 8 H100 GPUs under our profiling setting; additional annotator and runtime details are provided in Appendix A.

The core annotation process follows a hierarchical, global-to-local design. For each episode, we first query the VLM on the complete video to infer a high-level task plan and localize the temporal span of each candidate subtask. These video-level segments provide a global procedural structure, yielding more temporally consistent labels than independent frame-wise annotation while reducing redundant VLM calls. We then expand the segments into frame-wise subtask assignments and generate concise reasoning only on visually deduplicated keyframes. During post-processing, neighboring frames reuse nearby keyframe reasoning, preserving dense supervision without repeatedly annotating near-duplicate observations.

3.1.2 Data Scaling and Filtering

Using the annotation pipeline above, we generate frame-wise procedural annotations over diverse real-robot and simulation datasets, including DROID, BridgeData V2, Fractal, RH20T, Table30, selected subsets of OXE [7, 16, 31, 47, 59, 64], and common simulation benchmarks such as LIBERO, RoboTwin 2.0, and GR00T-Teleop-Sim [11, 39, 45]. Before annotation, we sample and screen candidate datasets to remove sources with poor visual quality or ambiguous instructions. A detailed list of preprocessed datasets is provided in Appendix B.

The resulting *ProcCorpus-60M* augments raw manipulation trajectories with procedure-level supervision. Each annotated sample contains task-centric scene reasoning, including a concise description of task-relevant visual evidence and a completion-state assignment; subtask annotations, including the current subtask and the global subtask structure of the trajectory; and optional target grounding annotations in the form of 2D bounding boxes. Together, these annotations connect visual observations with task progress, remaining steps, and manipulation targets. ProcCorpus-60M contains about 400K

real-robot and simulated trajectories with over 60M annotated frames, serving as the data foundation for procedure-aware pretraining on manipulation tasks.

Since automatic VLM annotation may still introduce noisy labels, we additionally curate a high-quality refinement subset. Through manual inspection, we select about 15K trajectories whose subtask segmentation closely matches the original videos while preserving task diversity. This curated refinement set is used in the refinement stage of ProcVLM training, where precise subtask alignment is more important than raw data scale.

3.2 Learning Procedure-Grounded Progress Rewards

Building on ProcCorpus-60M, we introduce *ProcVLM*, an embodied VLM for learning procedure-grounded progress rewards. Rather than deriving progress from elapsed time or terminal outcomes, ProcVLM defines continuous progress targets from frame-wise subtask annotations and predicts them through reasoning over the current execution stage and remaining steps. Since ProcCorpus-60M provides structured annotations rather than ready-to-use training samples, we further convert them into procedure-aware VQA tasks and jointly train ProcVLM for textual subtask reasoning and continuous progress estimation.

3.2.1 Procedure-Defined Progress Targets

To derive continuous progress labels from subtask annotations, we combine subtask-level structure with intra-subtask visual motion. Let T denote the trajectory length, K the number of valid subtasks, $k(t)$ the subtask index at time t, and $[s_k, e_k]$ the temporal span of the k-th subtask. We define progress as the normalized accumulation of local visual change weighted by subtask duration:

$$p(t) = \frac{\int_0^t w(\tau) r(\tau)\, d\tau}{\int_0^T w(\tau) r(\tau)\, d\tau},$$

where the subtask-level weight and the local progress rate are defined as

$$w(\tau) = \mathrm{clip}\left(\frac{K\left(e_{k(\tau)} - s_{k(\tau)}\right)}{T}, 0.75, 1.25\right), \qquad r(\tau) = \frac{\|\dot{\phi}(\tau)\|}{\int_{s_{k(\tau)}}^{e_{k(\tau)}} \|\dot{\phi}(u)\|\, du}.$$

Here $\phi(\cdot)$ denotes the visual representation of the observation. The weight $w(\tau)$ assigns each subtask a global progress budget according to its relative duration, while the clipping range serves as a soft anchor around an equal-subtask prior, preventing unusually long or short subtasks from dominating or vanishing. Since $r(\tau)$ is normalized within each subtask, it distributes this budget over time according to local visual changes. In implementation, we approximate $\|\dot{\phi}(\tau)\|$ with adjacent-frame perceptual differences and add a small numerical stabilizer for each frame, yielding a lightweight progress label that preserves subtask structure without reducing the target to linear interpolation over time.

3.2.2 Procedure-aware Pretraining

To turn the frame-wise annotations in ProcCorpus-60M into learnable supervision, we construct a procedure-aware VQA training set, termed *ProcVQA*, and formulate pretraining as multi-task VQA learning over robot observations and task instructions. Each training sample consists of a task instruction, one or more visual observations sampled from the trajectory, and a target response derived from the synthesized annotations. As illustrated in Figure 1, ProcVQA is centered on task progress estimation and further incorporates two auxiliary process-reasoning tasks to help the model understand task procedures beyond scalar progress prediction. Additional details of ProcVQA construction are provided in Appendix C.1.

Subtask-structured progress prediction. For progress prediction, we supervise the model to output the remaining atomic subtasks before estimating task completion. The final response ends with a continuous completion value wrapped by a structured progress tag: `<progress>` p `%</progress>`. This format grounds progress estimation in explicit task structure rather than superficial visual correlations, while also producing reasoning-formatted supervision for later adaptation.

Action segmentation. Given a task execution video and its task instruction, the model predicts a sequence of atomic subtask segments with semantic labels and temporal boundaries. This auxiliary

task supervises the model to identify stage transitions in long-horizon manipulation and builds temporal understanding of task procedures.

Future planning. Given recent observations and task instruction, the model predicts the executable subtasks required in subsequent steps. This auxiliary task focuses on connecting the current manipulation state with future procedural actions, enabling the model to recover the remaining task structure needed for planning and failure recovery.

3.2.3 ProcVLM Architecture and Training Objectives

ProcVLM is built on a compact vision-language backbone initialized from Qwen3-VL-2B-Instruct [4]. The backbone takes task instructions and visual observations as input, and generates task-specific textual responses through the standard autoregressive language modeling head. This branch supports action segmentation, future planning, and reasoning-formatted progress prediction.

To enable continuous progress prediction, we attach a progress value head on top of the shared VLM representations. For progress-estimation samples, this branch is activated to regress a scalar completion score from contextual hidden states produced by the backbone. Specifically, the value head applies attention pooling over relevant multimodal and textual representations before predicting the progress value. This gated design preserves the original language generation pathway while adding a dedicated continuous regression route for progress estimation, helping mitigate the tendency of token-based numerical prediction to collapse into coarse or quantized anchors [20, 42, 60, 72]. By sharing the backbone with textual subtask reasoning, the value head grounds its prediction in structured visual-language representations rather than isolated numeric cues.

ProcVLM is trained with a joint objective that couples text generation with continuous progress regression. Across all three task families, we apply the standard autoregressive language modeling loss to the supervised textual response. This also includes progress-estimation samples: their reasoning context and formatted answer are learned through the language modeling objective, while the scalar progress value parsed from the `<progress>` tag provides additional supervision for the value head.

For samples with progress supervision, the value branch predicts a continuous progress score $\hat{p}$, which is optimized against the ground-truth completion percentage p:

$$\mathcal{L}_{\text{value}} = \ell_{\text{reg}}(\hat{p}, p),$$

where ℓ_{reg} denotes the regression loss for continuous progress prediction. The final training objective is

$$\mathcal{L} = \mathcal{L}_{\text{LM}} + \lambda \cdot \mathbb{I}_{\text{prog}} \mathcal{L}_{\text{value}},$$

where $\mathcal{L}_{\text{LM}}$ is the standard autoregressive language modeling loss, $\mathbb{I}_{\text{prog}}$ indicates whether the sample contains progress supervision, and λ controls the weight of the value loss.

We train ProcVLM in two stages. The first stage uses the full ProcVQA dataset with approximately 20B tokens, while the second stage refines the model on a 2.8B-token curated set built from human-selected data to improve subtask alignment, procedural reasoning, and progress estimation. Further details on model configuration and training are provided in Appendices C.2 and C.3.

4 Experiments

We evaluate ProcVLM along three questions. *(Q1) Does ProcVLM improve embodied procedural understanding?* We evaluate this on ProcVQA, covering action segmentation, future planning, and task progress estimation. *(Q2) Can ProcVLM serve as a generalizable progress reward model?* We compare it with representative robotic reward models, test one-shot adaptation on RoboFAC, and conduct ablations on procedure-aware pretraining and reasoning-formatted supervision. *(Q3) Can ProcVLM improve downstream policy learning?* We use it for reward-guided fine-tuning and compare against vanilla supervised fine-tuning in simulation and real-robot settings.

4.1 Embodied Procedural Understanding

Setup. We evaluate ProcVLM on ProcVQA, a human-selected subtask-based VQA benchmark for embodied procedural understanding. ProcVQA includes three tasks: action segmentation, future planning, and task progress estimation. The ID split is derived from training-domain datasets such as

Table 1: Embodied procedural understanding evaluation on ProcVQA. We compare ProcVLM with mainstream VLMs on ID and OOD splits across action segmentation, future planning, and task progress estimation.

		Action Segmentation		Future Planning	Progress Estimation	
Dataset	Name	BF1@5 ↑	mMAE ↓	Success ↑	VOC ↑	EPR@50 ↑
ID	Qwen3VL	0.3945	0.6033	0.6034	0.4420	4.4645
	Qwen3.5	0.4296	0.7247	0.7931	0.5475	4.8741
	Gemini 3.1 Pro	0.4698	0.6578	0.7413	0.5087	5.3725
	GPT-5.4	0.5221	**0.4864**	0.5517	0.5229	4.8058
	ProcVLM (Ours)	**0.6924**	0.6758	**0.8103**	**0.8058**	**5.7489**
OOD	Qwen3VL	0.4330	0.6263	0.6896	0.5361	4.9178
	Qwen3.5	0.4185	**0.5131**	0.7758	0.6397	5.0932
	Gemini 3.1 Pro	0.4584	0.6015	0.7241	0.5987	6.0399
	GPT-5.4	0.4941	0.5158	0.5862	0.6553	5.2751
	ProcVLM (Ours)	**0.5802**	0.5579	**0.8448**	**0.7282**	**6.1152**

DROID, Bridge, Table30, and OXE, while the OOD split is constructed from unseen RoboTwin tasks. We compare ProcVLM with mainstream VLMs, including GPT-5.4, Gemini 3.1 Pro, Qwen3VL, and Qwen3.5. We report BF1@5/mMAE for action segmentation [27], human-evaluated Success for future planning, and VOC/EPR@50 for task progress estimation, with detailed metric definitions in Appendix D.

Findings. Table 1 shows that ProcVLM achieves the strongest overall performance on both ID and OOD splits. It obtains the best BF1@5, Success, VOC, and EPR@50, showing consistent gains in action segmentation, future planning, and progress estimation. Although its mMAE is not the lowest, ProcVLM remains comparable on this auxiliary localization metric while clearly improving the primary segmentation metric.

4.2 Generalizable Progress Reward Evaluation

4.2.1 Zero-Shot Comparison with Robotic Reward Models

Baselines and setup. We compare ProcVLM with Robometer and RoboDopamine on the ProcVQA progress estimation subset under a zero-shot setting. *Robometer* is trained on RBM-1M, a reward-learning dataset with over one million trajectories, which is larger in trajectory scale than our 400K-trajectory ProcCorpus-60M. It learns frame-level progress and success prediction together with trajectory-comparison preference learning, but does not explicitly supervise textual task-step reasoning over current and remaining subtasks. *RoboDopamine* learns step-aware process rewards from multi-view inputs through step-wise reward discretization and multi-perspective reward fusion, using initial, goal, before, and after states to predict relative progress hops. Our primary evaluation uses shuffled local-window queries: each query contains a short frame window, and windows from the same trajectory are evaluated independently without chronological ordering. For RoboDopamine, which requires before–after style inputs, we prepend a blank image to each local window as a neutral start anchor. Details of baseline adaptation are provided in Appendix E.

Findings. Table 2 shows that ProcVLM achieves the best VOC under the shuffled local-window setting on the OOD split of ProcVQA. Robometer attains higher EPR@50, likely due to its carefully designed progress-regression head, but its lower VOC indicates weaker trajectory-internal progress ordering under zero-shot evaluation. RoboDopamine remains relatively stable with a blank contrastive anchor and shuffled VQA-window evaluation, but still lags behind ProcVLM in VOC. Its lower EPR@50 suggests that, without a dedicated mechanism for continuous numerical regression, VLM-style progress prediction may still collapse toward coarse value anchors, a known challenge when continuous quantities are represented through discrete language tokens [20, 60, 69].

4.2.2 One-Shot Generalization

Setup and metrics. We evaluate one-shot generalization on the real-robot subset of RoboFAC, a failure-centric robotic VQA benchmark that evaluates task understanding, failure diagnosis, and

Table 2: Zero-shot comparison with robotic reward models on the ProcVQA-OOD progress estimation subset.

Dataset	Metric	ProcVLM-2B	Robometer-4B	RoboDopamine-4B
ProcVQA-OOD	VOC ↑	**0.7282**	0.5296	<u>0.7156</u>
	EPR@50 ↑	<u>6.1152</u>	**6.7212**	4.5693

correction planning from successful and failed manipulation executions [66]. We adapt it to evaluate reward models on both successful and failed executions. The 1-shot (Succ.) setting provides one successful demonstration per task, while 1-shot (Succ. + Fail.) additionally includes one demonstration for each task-specific failure type. Since one-shot demonstrations do not cover all camera views, the test set may contain unseen viewpoints, making the setting closer to real deployment. We report $\mathrm{VOC}_{\mathrm{succ}}$ for progress ordering, $\mathrm{MAE}_{\mathrm{fail}}$ for fault localization, MCC for binary success detection, and inference latency for full-trajectory progress evaluation. Details are in Appendix F.

Table 3: One-shot generalization on the real-robot subset of RoboFAC. We compare zero-shot and one-shot settings, where 1-shot (Succ.) provides one successful demonstration per task and 1-shot (Succ. + Fail.) additionally provides one demonstration for each task-specific failure type.

Setting	Name	$\mathrm{VOC}_{\mathrm{succ}}$ ↑	$\mathrm{MAE}_{\mathrm{fail}}$ ↓	MCC ↑	Latency (s) ↓
Zero-shot	Qwen3.5-27B	0.6191	0.1267	0.5678	1188
	RoboDopamine-8B	0.9095	0.2268	0.5506	157
	Robometer-4B	0.4129	<u>0.1218</u>	0.4516	139
	ProcVLM-2B	0.4920	0.2001	0.6665	**50**
1-shot (Succ.)	Robometer-4B	0.4094	0.1222	0.4435	136
	ProcVLM-2B	<u>0.9137</u>	0.1241	<u>0.7918</u>	81
1-shot (Succ. + Fail.)	Robometer-4B	0.4040	0.1221	0.4488	144
	ProcVLM-2B	**0.9301**	**0.1187**	**0.8053**	<u>80</u>

Fast one-shot adaptation. Table 3 shows that ProcVLM can rapidly adapt to unseen RoboFAC tasks with few demonstrations. While the zero-shot ProcVLM does not yet achieve the best progress ordering, adding a single successful trajectory per task improves $\mathrm{VOC}_{\mathrm{succ}}$ by 85.7%, increases MCC by 18.8%, and reduces $\mathrm{MAE}_{\mathrm{fail}}$ by 38.0%. Although the $\mathrm{VOC}_{\mathrm{succ}}$ gain may partly reflect adaptation to the successful demonstration, the simultaneous improvements on $\mathrm{MAE}_{\mathrm{fail}}$ and MCC indicate that semantic process reasoning enables ProcVLM to generalize task progress perception beyond the demonstrated success case. In contrast, Robometer shows little improvement under the same one-shot setting, suggesting that sparse preference supervision is less effective for single-demonstration adaptation. ProcVLM also maintains substantially lower inference latency than larger reward-model baselines, benefiting from its more compact model size.

Robust success detection and fault localization. Thanks to subtask-structured reasoning, zero-shot ProcVLM already achieves the best MCC among comparable settings, indicating stronger robustness in binary success detection. This advantage is further amplified when successful and failed demonstrations are introduced. For fault localization, although zero-shot ProcVLM is not the strongest, adapting it with only one successful demonstration brings $\mathrm{MAE}_{\mathrm{fail}}$ to a level comparable with Robometer, despite Robometer having a larger model scale and more extensive reward pretraining.

4.2.3 Ablation Studies

Ablation setup. We conduct the ablation study under the RoboFAC 1-shot (Succ.) setting described in Section 4.2.2. Variants without ProcVLM pretraining are initialized from Qwen3-VL-2B-Instruct, the same backbone used before our procedure-aware pretraining.

Procedure-aware pretraining enables one-shot transfer. Table 4 shows that removing ProcVLM pretraining causes the largest degradation, especially on $\mathrm{MAE}_{\mathrm{fail}}$ and MCC. This indicates that large-scale procedure-aware pretraining provides transferable representations for task progress perception, enabling the model to adapt to unseen RoboFAC tasks from only one successful demonstration. The

Table 4: Ablation study on RoboFAC 1-shot adaptation. *PT* denotes ProcVLM pretraining and *Rsn* denotes reasoning-formatted supervision. Δ reports relative performance changes from ProcVLM, where negative values indicate degradation after accounting for metric direction.

Variant	PT	Rsn.	$\text{VOC}_{\text{succ}} \uparrow$		$\text{MAE}_{\text{fail}} \downarrow$		MCC $\uparrow$	
			Score	Δ	Score	Δ	Score	Δ
ProcVLM	✓	✓	**0.9137**	–	**0.1241**	–	**0.7918**	–
w/o Reasoning	✓	–	0.9040	-1.1%	0.1284	-3.5%	0.7598	-4.0%
w/o Pretrain	–	✓	0.8281	-9.4%	0.2343	-88.8%	0.5949	-24.9%
Base	–	–	0.8795	-3.7%	0.1985	-60.0%	0.6854	-13.4%

large drop of the w/o Pretrain variant further shows that few-shot adaptation alone cannot compensate for the absence of procedure-aware pretraining.

Subtask-structured reasoning improves progress alignment. Removing reasoning-formatted supervision leads to consistent drops across all metrics, although the degradation is milder than removing pretraining. This suggests that explicit task-structure reasoning helps the pretrained model align low-level subtask cues with downstream progress labels, further improving one-shot generalization.

The Base variant further supports this conclusion. It still learns reasonable progress ordering from the provided successful demonstrations, but its severe degradation on MAE_{fail} and MCC reveals overfitting to surface-level progress cues. This highlights the strong synergy between procedure-aware pretraining and subtask-structured reasoning in improving one-shot transferability.

4.3 Reward Fine-tuning

Setup. We evaluate ProcVLM as a progress-based reward model for downstream policy learning. We build on SJTU Evo-RL, an open-source offline RL framework that supports value inference and advantage-conditioned policy training [13]. Using $\pi_{0.5}$ as the base policy [48], both SFT and RFT start from the same policy initialization and use the same training data. The SFT baseline follows the standard supervised fine-tuning pipeline. For RFT, ProcVLM assigns progress scores as dense rewards to training trajectories, which are used by Evo-RL to estimate advantages within a 50-step horizon. Within each task, the top 30% advantage samples are labeled as positive and the remaining samples are labeled as negative, serving as auxiliary conditions during reward-guided fine-tuning. Experiments are conducted on LIBERO-10 simulation tasks [39] and a real-robot stack-bowls task in our locally deployed JAKA environment. Further real-robot details are provided in Appendix G.

Table 5: Reward fine-tuning results in simulation and real-robot settings. Values are task success rates (%, ↑); Δ denotes percentage-point gain over SFT; steps use the same batch size across methods.

(a) Simulation: LIBERO-10

Steps	SFT	RFT	Δ
1000	73.2	73.6	+0.4
2000	72.8	74.0	+1.2

(b) Real Robot: Stack Bowls

Steps	SFT	RFT	Δ
5k	37.5	62.5	+25.0
10k	70.8	83.3	+12.5

Findings. Table 5 shows that ProcVLM-guided RFT improves over vanilla SFT in both simulation and real-robot settings. On LIBERO-10, RFT yields moderate early-stage gains of +0.4 and +1.2 points at 1K and 2K steps, as both methods start from the same strong pretrained policy. On the real-robot stack-bowls task, RFT brings larger gains of +25.0 and +12.5 points at 5K and 10K steps. This larger effect is expected because the teleoperation-collected real-robot data contain noisier local behaviors, such as repeated grasp retries, which standard SFT may overfit to during early training. By using ProcVLM rewards for advantage estimation and conditioning updates on high-advantage samples, RFT downweights less useful segments and stabilizes policy fine-tuning.

5 Conclusion

This work presents ProcVLM as a procedure-grounded reward model for robotic learning. By combining subtask-structured progress supervision with reasoning over remaining steps, ProcVLM provides discriminative feedback beyond time interpolation. Our results suggest procedure-aware pretraining is a promising route toward transferable reward models for reward-guided policy optimization.

Limitations. ProcVLM learns progress from procedure-defined supervision, so its estimates can be affected by the quality of subtask decomposition and temporal boundary localization [28, 33, 53]. Our downstream experiments focus on a limited set of reward-guided optimization settings, rather than exhaustively covering all possible integrations with policy-gradient-based reinforcement learning and preference-based optimization [12, 52, 55]. ProcVLM uses a lightweight progress-regression head, leaving calibration and robustness improvements to future work through distributional, stronger regression, or comparison-based objectives [5, 37, 58].

References

[1] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, Alexander Herzog, et al. Do as i can, not as i say: Grounding language in robotic affordances, 2022.

[2] Ali Amin, Raichelle Aniceto, Ashwin Balakrishna, Kevin Black, Ken Conley, Grace Connors, James Darpinian, Karan Dhabalia, Jared DiCarlo, Danny Driess, Michael Equi, Adnan Esmail, Yunhao Fang, Chelsea Finn, Catherine Glossop, Thomas Godden, Ivan Goryachev, Lachy Groom, Hunter Hancock, Karol Hausman, Gashon Hussein, Brian Ichter, Szymon Jakubczak, Rowan Jen, Tim Jones, Ben Katz, Liyiming Ke, Chandra Kuchi, Marinda Lamb, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Yao Lu, Vishnu Mano, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Charvi Sharma, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, Will Stoeckle, Alex Swerdlow, James Tanner, Marcel Torne, Quan Vuong, Anna Walling, Haohuan Wang, Blake Williams, Sukwon Yoo, Lili Yu, Ury Zhilinsky, and Zhiyuan Zhou. $\pi^*_{0.6}$: A vla that learns from experience. *arXiv preprint arXiv:2511.14759*, 2025.

[3] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. In *Advances in Neural Information Processing Systems*, 2017.

[4] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, Mei Li, Kaixin Li, Zicheng Lin, Junyang Lin, Xuejing Liu, Jiawei Liu, Chenglong Liu, Yang Liu, Dayiheng Liu, Shixuan Liu, Dunjie Lu, Ruilin Luo, Chenxu Lv, Rui Men, Lingchen Meng, Xuancheng Ren, Xingzhang Ren, Sibo Song, Yuchong Sun, Jun Tang, Jianhong Tu, Jianqiang Wan, Peng Wang, Pengfei Wang, Qiuyue Wang, Yuxuan Wang, Tianbao Xie, Yiheng Xu, Haiyang Xu, Jin Xu, Zhibo Yang, Mingkun Yang, Jianxin Yang, An Yang, Bowen Yu, Fei Zhang, Hang Zhang, Xi Zhang, Bo Zheng, Humen Zhong, Jingren Zhou, Fan Zhou, Jing Zhou, Yuanzhi Zhu, and Ke Zhu. Qwen3-vl technical report, 2025. URL `https://arxiv.org/abs/2511.21631`.

[5] Marc G. Bellemare, Will Dabney, and Rémi Munos. A distributional perspective on reinforcement learning. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 449–458. PMLR, 2017.

[6] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0: A vision-language-action flow model for general robot control, 2024. URL `https://arxiv.org/abs/2410.24164`.

[7] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alexander Herzog, Jasmine Hsu, et al. Rt-1: Robotics transformer for real-world control at scale, 2022. URL `https://arxiv.org/abs/2212.06817`.

[8] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control, 2023. URL `https://arxiv.org/abs/2307.15818`.

[9] Yevgen Chebotar, Quan Vuong, Karol Hausman, Fei Xia, Yao Lu, Alex Irpan, Aviral Kumar, Tianhe Yu, Alexander Herzog, Karl Pertsch, Keerthana Gopalakrishnan, Julian Ibarz, Ofir Nachum, Sumedh Anand Sontakke, Grecia Salazar, Huong T. Tran, Jodilyn Peralta, Clayton Tan, Deeksha Manjunath, Jaspiar Singh, Brianna Zitkovich, Tomas Jackson, Kanishka Rao, Chelsea Finn, and Sergey Levine. Q-transformer: Scalable offline reinforcement learning via autoregressive q-functions. In *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 3909–3928. PMLR, 2023.

[10] Shirui Chen, Cole Harrison, Ying-Chun Lee, Angela Jin Yang, Zhongzheng Ren, Lillian J. Ratliff, Jiafei Duan, Dieter Fox, and Ranjay Krishna. Topreward: Token probabilities as hidden zero-shot rewards for robotics, 2026.

[11] Tianxing Chen, Zanxin Chen, Baijun Chen, Zijian Cai, Yibin Liu, Zixuan Li, Qiwei Liang, Xianliang Lin, Yiheng Ge, Zhenyu Gu, Weiliang Deng, Yubin Guo, Tian Nian, Xuanbing Xie, Qiangyu Chen, Kailun Su, Tianling Xu, Guodong Liu, Mengkang Hu, Huan ang Gao, Kaixuan Wang, Zhixuan Liang, Yusen Qin, Xiaokang Yang, Ping Luo, and Yao Mu. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation, 2025. URL https://arxiv.org/abs/2506.18088.

[12] Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems*, 2017.

[13] Evo-RL Contributors. Evo-rl: Towards iterative policy improvement in real-world offline rl. https://github.com/MINT-SJTU/Evo-RL, 2026.

[14] Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, et al. Palm-e: An embodied multimodal language model, 2023. URL https://arxiv.org/abs/2303.03378.

[15] Yuqing Du, Mengjiao Yang, Pete Florence, Fei Xia, Ayzaan Wahid, Brian Ichter, Pierre Sermanet, Tianhe Yu, et al. Vision-language models as success detectors, 2023.

[16] Hao-Shu Fang, Hongjie Fang, Zhenyu Tang, Jirong Liu, Chenxi Wang, Junbo Wang, Haoyi Zhu, and Cewu Lu. Rh20t: A comprehensive robotic dataset for learning diverse skills in one-shot, 2023. URL https://arxiv.org/abs/2307.00595.

[17] Senyu Fei, Siyin Wang, Li Ji, Ao Li, Shiduo Zhang, Liming Liu, Jinlong Hou, Jingjing Gong, Xianzhong Zhao, and Xipeng Qiu. Srpo: Self-referential policy optimization for vision-language-action models. *arXiv preprint arXiv:2511.15605*, 2025.

[18] Gemini Robotics Team, Abbas Abdolmaleki, Saminda Abeyruwan, Joshua Ainslie, Jean-Baptiste Alayrac, et al. Gemini robotics 1.5: Pushing the frontier of generalist robots with advanced embodied reasoning, thinking, and motion transfer, 2025.

[19] Gemini Robotics Team, Saminda Abeyruwan, Joshua Ainslie, Jean-Baptiste Alayrac, et al. Gemini robotics: Bringing ai into the physical world, 2025.

[20] Siavash Golkar, Mariel Pettee, Michael Eickenberg, Alberto Bietti, Miles Cranmer, Geraud Krawezik, Francois Lanusse, Michael McCabe, Ruben Ohana, Liam Holden Parker, Bruno Régaldo-Saint Blancard, Tiberiu Tesileanu, Kyunghyun Cho, and Shirley Ho. xval: A continuous number encoding for large language models, 2023.

[21] Google DeepMind. Gemini robotics-er 1.5. Model card and technical report, 2025. URL https://deepmind.google/models/gemini-robotics/gemini-robotics-er/.

[22] Dongchi Huang, Zhirui Fang, Tianle Zhang, Yihang Li, Lin Zhao, and Chunhe Xia. Co-rft: Efficient fine-tuning of vision-language-action models through chunked offline reinforcement learning. *arXiv preprint arXiv:2508.02219*, 2025.

[23] Wenlong Huang, Chen Wang, Ruohan Zhang, Yunzhu Li, Jiajun Wu, and Li Fei-Fei. Voxposer: Composable 3d value maps for robotic manipulation with language models. In *Conference on Robot Learning*, 2023.

[24] Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, et al. Inner monologue: Embodied reasoning through planning with language models. In *Proceedings of the Conference on Robot Learning*, 2023.

[25] Wenlong Huang, Igor Mordatch, Deepak Pathak, et al. Rekep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation, 2024.

[26] Kuan-Hao Hung, Po-Chen Lo, Jen-Feng Yeh, et al. Victor: Learning hierarchical vision-instruction correlation rewards for long-horizon manipulation, 2025.

[27] Yuchi Ishikawa, Seito Kasai, Yoshimitsu Aoki, and Hirokatsu Kataoka. Alleviating over-segmentation errors by detecting action boundaries. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2021.

[28] Yuheng Ji, Huajie Tan, Jiayu Shi, Xiaoshuai Hao, Yuan Zhang, Hengyuan Zhang, Pengwei Wang, Mengdi Zhao, Yao Mu, Pengju An, et al. Robobrain: A unified brain model for robotic manipulation from abstract to concrete, 2025. URL `https://arxiv.org/abs/2502.21257`.

[29] Yunfan Jiang, Agrim Gupta, Zichen Zhang, Guanzhi Wang, Yifeng Dou, Yanjie Chen, Li Fei-Fei, Anima Anandkumar, Yuke Zhu, and Linxi Fan. Vima: General robot manipulation with multimodal prompts. In *International Conference on Machine Learning*, 2023.

[30] Xinyao Kang and Yi-Ling Kuo. Incorporating task progress knowledge for subgoal generation in robotic manipulation through image edits. In *Proceedings of the Winter Conference on Applications of Computer Vision*, pages 7490–7499, 2025.

[31] Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, Peter David Fagan, Joey Hejna, Masha Itkina, Marion Lepert, Yecheng Jason Ma, Patrick Tree Miller, Jimmy Wu, Suneel Belkhale, Shivin Dass, Huy Ha, Arhan Jain, Abraham Lee, Youngwoon Lee, Marius Memmel, Sungjae Park, Ilija Radosavovic, Kaiyuan Wang, Albert Zhan, Kevin Black, Cheng Chi, Kyle Beltran Hatch, Shan Lin, Jingpei Lu, Jean Mercat, Abdul Rehman, Pannag R Sanketi, Archit Sharma, Cody Simpson, Quan Vuong, Homer Rich Walke, Blake Wulfe, Ted Xiao, Jonathan Heewon Yang, Arefeh Yavary, Tony Z. Zhao, Christopher Agia, Rohan Baijal, Mateo Guaman Castro, Daphne Chen, Qiuyu Chen, Trinity Chung, Jaimyn Drake, Ethan Paul Foster, Jensen Gao, Vitor Guizilini, David Antonio Herrera, Minho Heo, Kyle Hsu, Jiaheng Hu, Muhammad Zubair Irshad, Donovon Jackson, Charlotte Le, Yunshuang Li, Kevin Lin, Roy Lin, Zehan Ma, Abhiram Maddukuri, Suvir Mirchandani, Daniel Morton, Tony Nguyen, Abigail O'Neill, Rosario Scalise, Derick Seale, Victor Son, Stephen Tian, Emi Tran, Andrew E. Wang, Yilin Wu, Annie Xie, Jingyun Yang, Patrick Yin, Yunchu Zhang, Osbert Bastani, Glen Berseth, Jeannette Bohg, Ken Goldberg, Abhinav Gupta, Abhishek Gupta, Dinesh Jayaraman, Joseph J Lim, Jitendra Malik, Roberto Martín-Martín, Subramanian Ramamoorthy, Dorsa Sadigh, Shuran Song, Jiajun Wu, Michael C. Yip, Yuke Zhu, Thomas Kollar, Sergey Levine, and Chelsea Finn. Droid: A large-scale in-the-wild robot manipulation dataset, 2025. URL `https://arxiv.org/abs/2403.12945`.

[32] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Pannag Sanketi, Quan Vuong, et al. Openvla: An open-source vision-language-action model, 2024. URL `https://arxiv.org/abs/2406.09246`.

[33] Colin Lea, Michael D. Flynn, Rene Vidal, Austin Reiter, and Gregory D. Hager. Temporal convolutional networks for action segmentation and detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 156–165, 2017.

[34] Tony Lee, Andrew Wagenmaker, Karl Pertsch, Percy Liang, Sergey Levine, and Chelsea Finn. Roboreward: General-purpose vision-language reward models for robotics, 2026.

[35] Peng Li, Yixiao Chen, Hao Wu, et al. Bridgevla: Input-output alignment for efficient 3d manipulation learning with vision-language models, 2025.

[36] Qiyang Li, Zhiyuan Zhou, and Sergey Levine. Reinforcement learning with action chunking. *arXiv preprint arXiv:2507.07969*, 2025.

[37] Anthony Liang, Yigit Korkmaz, Jiahui Zhang, Minyoung Hwang, Abrar Anwar, Sidhant Kaushik, Aditya Shah, Alex S. Huang, Luke Zettlemoyer, Dieter Fox, Yu Xiang, Anqi Li, Andreea Bobu, Abhishek Gupta, Stephen Tu, Erdem Biyik, and Jesse Zhang. Robometer: Scaling general-purpose robotic reward models via trajectory comparisons, 2026.

[38] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation*, 2023.

[39] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning, 2023. URL https://arxiv.org/abs/2306.03310.

[40] Fangchen Liu, Kuan Fang Liu, Puchang Xie, et al. Moka: Open-world robotic manipulation through mark-based visual prompting, 2024.

[41] Guanxing Lu, Wenkai Guo, Chubin Zhang, Yuheng Zhou, Haonan Jiang, Zifeng Gao, Yansong Tang, and Ziwei Wang. Vla-rl: Towards masterful and general robotic manipulation with scalable reinforcement learning, 2025.

[42] Xingyuan Ma, Shuai He, Anlong Ming, Haobin Zhong, and Huadong Ma. Regression over classification: Assessing image aesthetics via multimodal large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 7827–7835, 2026. doi: 10.1609/aaai.v40i10.37726.

[43] Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-level reward design via coding large language models, 2023.

[44] Yecheng Jason Ma, Joey Hejna, Ayzaan Wahid, Chuyuan Fu, Dhruv Shah, Jacky Liang, Zhuo Xu, Sean Kirmani, Peng Xu, Danny Driess, Ted Xiao, Jonathan Tompson, Osbert Bastani, Dinesh Jayaraman, Wenhao Yu, Tingnan Zhang, Dorsa Sadigh, and Fei Xia. Vision language models are in-context value learners, 2024. URL https://arxiv.org/abs/2411.04549.

[45] NVIDIA, Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, et al. Gr00t n1: An open foundation model for generalist humanoid robots, 2025. URL https://arxiv.org/abs/2503.14734.

[46] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy, 2024. URL https://arxiv.org/abs/2405.12213.

[47] Open X-Embodiment Collaboration, Abby O'Neill, Abdul Rehman, Abhinav Gupta, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, et al. Open x-embodiment: Robotic learning datasets and rt-x models, 2023. URL https://arxiv.org/abs/2310.08864.

[48] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization, 2025. URL https://arxiv.org/abs/2504.16054.

[49] Delin Qu, Haoming Song, Qizhi Chen, et al. Spatialvla: Exploring spatial representations for visual-language-action model, 2025.

[50] Scott Reed, Konrad Zolna, Emilio Parisotto, Sergio Gomez Colmenarejo, Alexander Novikov, Gabriel Barth-Maron, Mai Gimenez, Yury Sulsky, Jackie Kay, Jost Tobias Springenberg, et al. A generalist agent, 2022.

[51] Allen Z. Ren, Aakarsh Dixit, Anna Bodrova, Anikait Singh, Stephen Tu, Noah Brown, Peng Xu, Fei Xia, Ted Xiao, Sergey Levine, et al. Robopoint: A vision-language model for spatial affordance prediction for robotics, 2024.

[52] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[53] Fadime Sener and Angela Yao. Unsupervised learning and segmentation of complex activities from video. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 8368–8376, 2018.

[54] Pierre Sermanet, Tianli Ding, Jeffrey Zhao, Fei Xia, Debidatta Dwibedi, Keerthana Gopalakrishnan, Christine Chan, Gabriel Dulac-Arnold, Sharath Maddineni, Nikhil J. Joshi, et al. Robovqa: Multimodal long-horizon reasoning for robotics, 2023. URL https://arxiv.org/abs/2311.00899.

[55] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[56] Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. Progprompt: Generating situated robot task plans using large language models, 2023.

[57] Sumedh A. Sontakke, Jesse Zhang, Sebastian M. R. Arnold, Karl Pertsch, Erdem Biyik, Dorsa Sadigh, Chelsea Finn, and Laurent Itti. Roboclip: One demonstration is enough to learn robot policies, 2023.

[58] Huajie Tan, Sixiang Chen, Yijie Xu, Zixiao Wang, Yuheng Ji, Cheng Chi, Yaoxu Lyu, Zhongxia Zhao, Xiansheng Chen, Peterson Co, Shaoxuan Xie, Guocai Yao, Pengwei Wang, Zhongyuan Wang, and Shanghang Zhang. Robo-dopamine: General process reward modeling for high-precision robotic manipulation, 2025.

[59] Homer Walke, Kevin Black, Abraham Lee, Moo Jin Kim, Max Du, Chongyi Zheng, Tony Zhao, Philippe Hansen-Estruch, Quan Vuong, Andre He, Vivek Myers, Kuan Fang, Chelsea Finn, and Sergey Levine. Bridgedata v2: A dataset for robot learning at scale. In *Conference on Robot Learning (CoRL)*, 2023.

[60] Pei Wang, Zhaowei Cai, Hao Yang, Davide Modolo, and Ashwin Swaminathan. Enhancing numerical prediction of mllms with soft labeling. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 3424–3434, October 2025.

[61] Weiyun Wang, Zhangwei Gao, Lixin Gu, Hengjun Pu, Long Cui, Xingguang Wei, Zhaoyang Liu, Linglin Jing, Shenglong Ye, Jie Shao, Zhaokai Wang, Zhe Chen, Hongjie Zhang, Ganlin Yang, Haomin Wang, Qi Wei, Jinhui Yin, Wenhao Li, Erfei Cui, Guanzhou Chen, Zichen Ding, Changyao Tian, Zhenyu Wu, Jingjing Xie, Zehao Li, Bowen Yang, Yuchen Duan, Xuehui Wang, Zhi Hou, Haoran Hao, Tianyi Zhang, Songze Li, Xiangyu Zhao, Haodong Duan, Nianchen Deng, Bin Fu, Yinan He, Yi Wang, Conghui He, Botian Shi, Junjun He, Yingtong Xiong, Han Lv, Lijun Wu, Wenqi Shao, Kaipeng Zhang, Huipeng Deng, Biqing Qi, Jiaye Ge, Qipeng Guo, Wenwei Zhang, Songyang Zhang, Maosong Cao, Junyao Lin, Kexian Tang, Jianfei Gao, Haian Huang, Yuzhe Gu, Chengqi Lyu, Huanze Tang, Rui Wang, Haijun Lv, Wanli Ouyang, Limin Wang, Min Dou, Xizhou Zhu, Tong Lu, Dahua Lin, Jifeng Dai, Weijie Su, Bowen Zhou, Kai Chen, Yu Qiao, Wenhai Wang, and Gen Luo. Internvl3.5: Advancing open-source multimodal models in versatility, reasoning, and efficiency, 2025. URL `https://arxiv.org/abs/2508.18265`.

[62] Yanru Wu, Weiduo Yuan, Ang Qi, Vitor Guizilini, Jiageng Mao, and Yue Wang. Large reward models: Generalizable online robot reward generation with vision-language models, 2026. URL `https://arxiv.org/abs/2603.16065`.

[63] Wenli Xiao, Haotian Lin, Andy Peng, Haoru Xue, Tairan He, Yuqi Xie, Fengyuan Hu, Jimmy Wu, Zhengyi Luo, Linxi Fan, Guanya Shi, and Yuke Zhu. Self-improving vision-language-action models with data generation via residual rl. *arXiv preprint arXiv:2511.00091*, 2025.

[64] Adina Yakefu, Bin Xie, Chongyang Xu, Enwen Zhang, Erjin Zhou, Fan Jia, Haitao Yang, Haoqiang Fan, Haowei Zhang, Hongyang Peng, Jing Tan, Junwen Huang, Kai Liu, Kaixin Liu, Kefan Gu, Qinglun Zhang, Ruitao Zhang, Saike Huang, Shen Cheng, Shuaicheng Liu, Tiancai Wang, Tiezhen Wang, Wei Sun, Wenbin Tang, Yajun Wei, Yang Chen, Youqiang Gui, Yucheng Zhao, Yunchao Ma, Yunfei Wei, Yunhuan Yang, Yutong Guo, Ze Chen, Zhengyuan Du, Ziheng Zhang, Ziming Liu, and Ziwei Yan. Robochallenge: Large-scale real-robot evaluation of embodied policies, 2025. URL `https://arxiv.org/abs/2510.17950`.

[65] Senqiao Yang, Hongyu Li, Bo Wang, et al. Instructvla: Vision-language-action instruction tuning from understanding to manipulation, 2025.

[66] Zewei Ye, Weifeng Lu, Minghao Ye, Tao Lin, Shuo Yang, Junchi Yan, and Bo Zhao. Robofac: A comprehensive framework for robotic failure analysis and correction, 2026. URL `https://arxiv.org/abs/2505.12224`.

[67] Silong Yong, Stephen Sheng, Carl Qi, Xiaojie Wang, Evan Sheehan, Anurag Shivaprasad, Yaqi Xie, Katia Sycara, and Yesh Dattatreya. Generalizable dense reward for long-horizon robotic tasks, 2026. URL https://arxiv.org/abs/2604.00055.

[68] Yifu Yuan, Haiqin Cui, Yaoting Huang, Yibin Chen, Fei Ni, Zibin Dong, Pengyi Li, Yan Zheng, and Jianye Hao. Embodied-r1: Reinforced embodied reasoning for general robotic manipulation, 2025.

[69] Jonas Zausinger, Lars Pennig, Anamarija Kozina, Sean Sdahl, Julian Sikora, Adrian Dendorfer, Timofey Kuznetsov, Mohamad Hagog, Nina Wiedemann, Kacper Chlodny, Vincent Limbach, Anna Ketteler, Thorben Prein, Vishwa Mohan Singh, Michael Morris Danziger, and Jannis Born. Regress, don't guess: A regression-like loss on number tokens for language models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.

[70] Shaopeng Zhai, Qi Zhang, Tianyi Zhang, Fuxian Huang, Haoran Zhang, Ming Zhou, Shengzhe Zhang, Litao Liu, Sixu Lin, and Jiangmiao Pang. A vision-language-action-critic model for robotic real-world reinforcement learning, 2025.

[71] Jiahui Zhang, Yusen Luo, Abrar Anwar, Sumedh Anand Sontakke, Joseph J. Lim, Jesse Thomason, Erdem Biyik, and Jesse Zhang. Rewind: Language-guided rewards teach robot policies without new demonstrations, 2025.

[72] Jianshu Zhang, Chengxuan Qian, Haosen Sun, Haoran Lu, Dingcheng Wang, Letian Xue, and Han Liu. PROGRESSLM: Towards progress reasoning in vision-language models, 2026.

[73] Zijian Zhang, Kaiyuan Zheng, Zhaorun Chen, Joel Jang, Yi Li, Siwei Han, Chaoqi Wang, Mingyu Ding, Dieter Fox, and Huaxiu Yao. GRAPE: Generalizing robot policy via preference alignment. *arXiv preprint arXiv:2411.19309*, 2024.

Appendix Contents

A Annotation Pipeline Details **18**

A.1 Annotator Models . . . 18

A.2 Pipeline Execution and Post-processing . . . 18

A.3 Pipeline Profiling . . . 18

A.4 Prompt Templates . . . 19

A.4.1 Plan Generation . . . 19

A.4.2 Subtask Segmentation . . . 20

A.4.3 Frame-level Reasoning . . . 20

B Preprocessed Dataset Details **21**

C ProcVLM Training Details **23**

C.1 ProcVQA Construction . . . 23

C.1.1 Construction from Procedural Annotations . . . 23

C.1.2 Task Templates . . . 23

C.1.3 Training Sample Statistics . . . 24

C.2 ProcVLM Configuration . . . 24

C.2.1 Backbone and Output Branches . . . 24

C.2.2 Progress-Value Gating . . . 25

C.2.3 Value-Head Pooling . . . 25

C.3 Training Pipeline and Implementation Details . . . 25

C.3.1 Language Modeling Supervision . . . 25

C.3.2 Progress Value Supervision . . . 26

C.3.3 Leakage-Prevention Tail Dropout . . . 26

C.3.4 Two-stage Training Pipeline . . . 26

C.3.5 Training Hyperparameters . . . 27

D ProcVQA Evaluation Details **27**

D.1 Benchmark and Splits . . . 27

D.2 Metrics . . . 27

E Reward Model Baseline Adaptation Details **28**

F RoboFAC Evaluation Details **29**

F.1 Implementation Details . . . 29

F.2 Metrics . . . 29

G Real-Robot Experiment Setup **30**

H Reward Modeling Cases **31**

A Annotation Pipeline Details

This section provides additional details of the annotator models, hierarchical annotation process, pipeline profiling results, and prompt templates used in Section 3.1.

A.1 Annotator Models

We use different large VLMs for different annotation stages. For video-level planning and subtask temporal localization, we use Qwen3-VL-235B-A22B-Instruct as the annotator model, since these tasks require long-context video understanding and global procedural reasoning. For frame-level reasoning and grounding annotations, we use InternVL3.5-38B, which provides efficient and reliable single-frame visual reasoning. All generated annotations are post-processed into JSONL files and used to construct procedure-aware VQA tasks for ProcVLM pretraining.

A.2 Pipeline Execution and Post-processing

The annotation pipeline is implemented as a queue-based asynchronous system to avoid serialized CPU/GPU execution. It consists of four decoupled modules: data reading, CPU-side preprocessing, GPU-side VLM inference, and post-processing. Each module runs independently and communicates through bounded queues, so that slow I/O, image preprocessing, and VLM inference do not block each other. This design overlaps CPU preparation with GPU execution, reduces pipeline bubbles, and keeps the annotator GPUs continuously supplied with ready-to-run batches.

Data reader. The data reader scans raw robot episodes, organizes them into task-consistent trajectories, and performs sparse video sampling according to the annotation stage. It also prefetches images and metadata asynchronously, including task instructions, camera keys, frame indices, and episode identifiers. This module isolates storage and decoding latency from the rest of the pipeline, preventing GPU workers from waiting on raw data loading.

CPU processor. The CPU processor converts sampled episodes into VLM-ready inputs. It performs image loading, resizing, multimodal preprocessing, prompt construction, and template formatting for different annotation tasks, including plan generation, subtask temporal localization, frame-level reasoning and target grounding. Because each annotation query often contains multiple images or video frames, we move image loading, multimodal preprocessing, and template formatting to CPU workers ahead of GPU inference, keeping ready-to-run batches available for GPU workers and reducing idle GPU time.

GPU worker. The GPU worker consumes preprocessed batches and runs large VLM inference. Depending on the annotation stage, it performs video-level task planning, subtask temporal localization, or keyframe-level reasoning and grounding. We use optimized inference backends such as vLLM and LMDeploy, together with local batched inference APIs, to keep large multimodal requests densely packed on the GPU and maximize annotation throughput.

Consumer and post-processing. The consumer parses VLM outputs, validates their format, and writes normalized JSONL annotations indexed by dataset name, episode id, frame id, and camera key. It expands video-level subtask segments into frame-wise assignments, propagates keyframe reasoning to neighboring non-keyframes when appropriate, checks temporal consistency for grounding box outputs and removes invalid boxes. This stage converts heterogeneous VLM outputs into a standardized annotation format, allowing different robot datasets to be merged into ProcCorpus and later converted into ProcVQA training samples.

Overall, the pipeline separates I/O-bound, CPU-bound, GPU-bound, and format-normalization workloads into independent stages. This modular design improves annotation throughput, reduces idle time caused by CPU/GPU synchronization, and makes large-scale frame-wise procedural annotation practical over hundreds of thousands of trajectories.

A.3 Pipeline Profiling

In our profiling experiment, the pipeline is deployed on 8 NVIDIA H100 GPUs with 80GB HBM3 memory. Under this setting, it processes about 4M keyframes per day. Figure 3 shows the throughput and GPU utilization of the annotation pipeline.

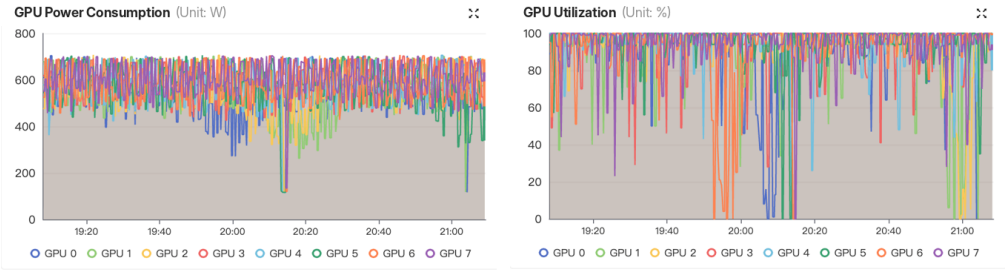


Figure 3: GPU power consumption and utilization over time across multiple GPUs during the annotation pipeline execution. The GPUs maintain consistently high utilization and stable power draw for most of the run, indicating efficient parallel workload scheduling with only brief transient drops.

A.4 Prompt Templates

We use task-specific prompt templates for plan generation, subtask temporal localization, and frame-level reasoning. The templates are shown below.

A.4.1 Plan Generation

```
I will give you a robot task and a video showing the robot arm performing the task.
You need to analyze actions of the robot arm from the video and decompose the task
into a sequence of detailed sub-tasks. The sequence of sub-tasks should lead to the
completion of the overall task.

###
Grasp [specific object]
e.g. "Grasp the red block"
Explain: Use this pattern when the robot isn't holding anything and needs to pick up
an object, before performing other actions like placing or lifting.
###
Place [specific object] onto / into [specific location]
e.g. "Place the cup onto the table" or "Place the screwdriver into the tool rack."
Explain: Use this pattern when the robot is holding an object and needs to put it
down at a specific location.
###
Push [specific object] [forward / backward / left / right]
e.g., "Push the blue block forward"
Explain: Use this pattern when the robot needs to move an object in a specific
direction by applying force to it, without lifting or grasping it. "Push" and
("Grasp", "Place") are mutually exclusive.
###
Tilt the gripper
e.g. "Tilt the gripper to pour the liquid" or "Tilt the gripper slightly to the left"
Explain: Use this pattern when the robot needs to adjust the angle of its gripper,
either to pour liquid or position the gripper for some specific task.
###
Hang [specific object] on / above [specific location]
e.g. "Hang the coat on the hook" or "Hang the cup above the table"
Explain: Use this pattern when the robot needs to suspend an object from a specific
location, such as hanging a coat on a hook or a cup above a table.
###
Press [specific object]
e.g., "Press the button" or "Press the power switch until it clicks."
###
Open [specific object]
e.g. "Open the door slowly"
###
Close [specific object]
e.g., "Close the lid securely"
```

```
###
Rotate [specific object]
e.g., "Rotate the knob clockwise"
###

All sub-tasks must be in exactly one of the patterns above, and should follow the
Explain for each pattern. There is no need to include the robot arm itself as an
object in the sub-tasks.

You should output sub-tasks in a numbered list format, starting from 1. Each line
contains one sub-task with a leading number and a period. No extra text or
explanation.

Task: {task}
Output:
```

A.4.2 Subtask Segmentation

```
You will be shown a VIDEO of a robot task and an UNORDERED list of planned sub-tasks.

Task: "{task}"
Planned sub-tasks:
{plans}

OBJECTIVE:
For each planned sub-task, if present, mark the frames where it starts and finishes.
If a sub-task is started but not finished, set complete_frame=null. If not present
at all, set both start_frame=null and complete_frame=null, and notes="not present".
If the video shows any action was interrupted and the overall task was not completed,
set overall_notes="task not completed".

OUTPUT FORMAT:
{
  "task": "<same as input task>",
  "subtasks": [
    {"id":1, "notes":"<<=60 words optional>",
     "start_frame":<int|null>, "complete_frame":<int|null>,
     "name":"<same text from plans>"},
    ...
  ],
  "overall_notes":"<<=30 words optional>"
}

HINTS:
- Find the changes in effector pose and object motion as candidate start/complete frames.
- The start frame can be picked slightly earlier and the complete frame slightly later
  to ensure the action is fully captured.
- If multiple candidates exist, pick the final success. If retries occur, record the
  final success.
- Use the last frame of the video as reference for overall_notes.

Now process the provided video and planned sub-tasks and return the JSON result ONLY.
```

A.4.3 Frame-level Reasoning

We use three reasoning templates depending on the task state: unfinished, finished, and give-up/failure. These templates are intentionally short and task-oriented, encouraging the annotator VLM to focus on task completion and remaining procedural steps rather than open-ended scene description.

Unfinished state.

```
The image shows a robot performing a task: '{task}', which may be incomplete.
Remaining subtasks: '{rest_sub_task}'.
Explain why it's unfinished and briefly describe the next steps based on image details.
```

```
Output (<=150 words, 3 sentences):
<analysis with image details>. This task is not finished <short reason>.
<one-sentence summary of next steps>.

Example:
Task: 'put all the green objects on the pink plate.'
Image: a green apple in robot arm, a green pear on blue plate.
Output:
Image shows a green apple held by the robot and a green pear on the blue plate.
This task is not finished because both green objects are not yet on the pink plate.
The robot should place the green apple on the pink plate, then move the green pear
from the blue plate to the pink plate.
```

Finished state.

```
The image shows a robot performing a task: '{task}', which is finished.
Explain briefly why it's completed based on image details.

Output (<=50 words, 2 sentences):
<analysis with image details>. This task is finished <short reason>.

Example:
Task: 'put all the green objects on the pink plate.'
Image: both green apple and pear on pink plate.
Output:
Image shows a green apple and pear on the pink plate. This task is finished because
all green objects are placed correctly.
```

Give-up or failed state.

```
The image shows a robot performing a task: '{task}', which is not finished.
Explain briefly why it's unfinished based on image details.

Output (<=50 words, 2 sentences):
<analysis with image details>. This task is not finished <short reason>.

Example:
Task: 'put all the green objects on the pink plate.'
Image: a green apple held by the robot, a green pear on blue plate.
Output:
Image shows a green apple in the robot arm and a green pear on the blue plate.
This task is not finished because neither object has been placed on the pink plate.
```

B Preprocessed Dataset Details

This section provides the detailed list of real-robot and simulation datasets preprocessed and annotated for ProcCorpus, as described in Section 3.1.2. We merge multiple task-level or configuration-level subsets belonging to the same dataset family, and report annotation statistics based only on subtask annotations.

Table 6 summarizes the dataset sources and subtask annotation coverage of ProcCorpus. In total, the corpus contains more than 60M raw frames from over 400K real-robot and simulated trajectories, with most datasets achieving high subtask annotation coverage. These statistics show that the annotation pipeline can scale to heterogeneous manipulation data while maintaining dense procedure-level supervision for ProcVLM pretraining.

As illustrated in Figure 4, each ProcCorpus annotation enriches a raw manipulation frame with task-centric reasoning, completion-state assignment, subtask structure, and target grounding.

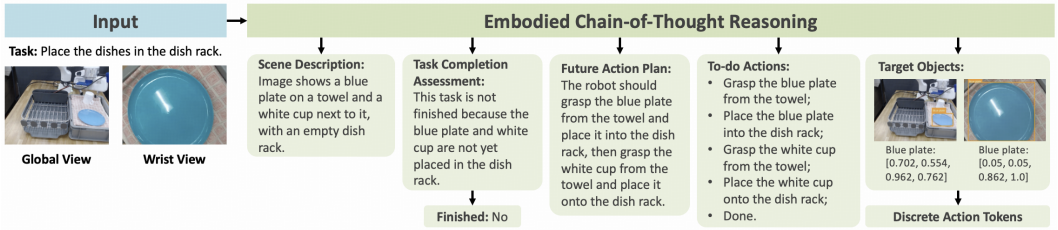


Figure 4: Example of *Embodied Chain-of-Thought (ECoT)* annotation in ProcCorpus. Given multi-view robot observations and a task instruction, ECoT enriches the raw frame with task-centric scene reasoning, completion assessment, future action planning, remaining to-do actions, target-object grounding, and optional discrete action tokens for VLA training.

Table 6: Datasets used for ProcCorpus construction. Annotated frames and annotation rates are computed based on subtask annotations only.

Dataset	Frames	Traj.	Ann. Frames	Ann. Rate
Real-robot datasets				
DROID	27,630,375	95,658	26,157,676	94.67%
BridgeData V2	2,863,587	78,637	2,790,172	97.44%
Fractal	3,786,400	87,212	3,778,827	99.80%
RH20T	1,699,138	4,433	1,621,815	95.45%
Table30	5,184,355	25,610	5,167,461	99.67%
OXE / Austin BUDS	34,112	50	34,112	100.00%
OXE / Austin SAILOR	353,094	240	353,094	100.00%
OXE / Austin SIRIUS	279,939	559	279,939	100.00%
OXE / Berkeley AUTOLab UR5	86,887	896	86,583	99.65%
OXE / Berkeley FANUC Manipulation	62,613	415	59,764	95.45%
OXE / CMU Play Fusion	235,922	576	235,096	99.65%
OXE / Columbia PushT	24,802	122	23,220	93.62%
OXE / DLR EDAN Shared Control	8,928	104	8,597	96.29%
OXE / DLR SARA Pour	12,971	100	12,771	98.46%
OXE / Dobb-E	1,139,911	5,208	1,087,931	95.44%
OXE / FMB	1,137,340	8,611	959,915	84.40%
OXE / IAMLab CMU Pickup-Insert	146,105	522	145,053	99.28%
OXE / Jaco Play	70,127	976	68,872	98.21%
OXE / NYU Door Opening	17,761	435	17,670	99.49%
OXE / QUT Dexterous Manipulation	176,278	200	175,538	99.58%
OXE / Stanford HYDRA	358,234	570	353,326	98.63%
OXE / TOTO	294,139	902	294,139	100.00%
OXE / UCSD Kitchen	3,970	150	3,491	87.93%
OXE / UT Austin Mutex	361,883	1,500	350,737	96.92%
Internal / JAKA Teleop	23,774	100	22,271	93.68%
Internal-xArm / Real-world Tabletop	366,152	2,005	366,152	100.00%
Real-robot total	**46,358,797**	**315,791**	**44,454,222**	**95.89%**
Simulation datasets				
GR00T-Teleop-Sim	5,820,277	24,000	5,807,497	99.78%
Internal-xArm / Simulation Tabletop	2,001,729	43,720	1,998,726	99.85%
LIBERO v2.1	273,465	1,693	273,465	100.00%
RoboTwin 2.0	6,012,086	27,200	5,997,357	99.76%
Simulation total	**14,107,557**	**96,613**	**14,077,045**	**99.78%**
Overall total	**60,466,354**	**412,404**	**58,531,267**	**96.80%**

C ProcVLM Training Details

This section provides additional training details for ProcVLM, including the construction of ProcVQA, the model configuration for progress-value prediction, and the two-stage training pipeline, as described in Section 3.2.

C.1 ProcVQA Construction

C.1.1 Construction from Procedural Annotations

ProcVQA is constructed from the frame-wise annotations in ProcCorpus. Each raw trajectory is represented by a task instruction and a robot execution video, where the video is treated as an ordered image sequence. The synthesized annotations provide subtask labels, temporal boundaries, frame-level procedural reasoning, and progress estimates.

We convert these annotations into supervised multimodal question-answering samples. Each sample consists of a textual query, one or more visual observations sampled from the trajectory, and a target response derived from the corresponding procedural annotations. This conversion turns dense frame-wise annotations into a unified VQA-style supervision format for multimodal instruction tuning.

We define an **atomic action** as an executable subtask that can be described by a **single explicit verb-level** action. A high-level task may admit multiple valid decompositions under this definition, but we regard them as acceptable as long as each atomic action is temporally coherent and verb-maximal, i.e., it covers a complete continuous execution phase whenever possible. Under this principle, different decompositions can still provide meaningful low-level and generalizable action descriptions.

For instance, the high-level instruction "clean the table" is not directly executable as a single verb-level action. When two tissues are on the table, a valid decomposition may include grasping the left tissue, placing it into the trash can, grasping the right tissue, and placing it into the trash can. ProcVQA therefore supervises the model to reason over explicit procedural steps rather than only imitate trajectory-level task descriptions.

C.1.2 Task Templates

These templates instantiate the pretraining tasks introduced in Section 3.2.2, with five concrete variants denoted as (a.1), (a.2), (b.1), (b.2), and (c). They use either video-level interleaved frame-index sequences for action segmentation or recent local observation windows for next-step prediction, remaining-step planning, and progress estimation.

An interleaved frame-index sequence represents a video as an ordered multimodal sequence, where each frame image is followed by its frame identifier, e.g., `<image><frame_id: 1><image><frame_id: 2>`. This format preserves the temporal order of sampled frames while remaining compatible with the image-text input interface of the VLM.

(a.1) Action segmentation with task instruction. Given an interleaved frame-index sequence and its task instruction, the model segments the execution into consecutive atomic actions:

```
Segment the execution of the task "{task}" into consecutive
atomic actions.  Each segment should correspond to a single,
explicit verb-level action.  Output a JSON list with keys
"action_description", "start_frame", and "end_frame".
```

The target response is a JSON-style list of action segments, where each segment contains an action description and its start and end frames.

(a.2) Task-free action segmentation. We also construct a task-free segmentation variant, where the model receives only the visual sequence and predicts the atomic action segments without the original task instruction:

```
Segment the actions shown in the image sequence into consecutive
atomic actions, each described by a single explicit verb.  Output
a JSON list with keys "action_description", "start_frame", and
"end_frame".
```

This variant encourages the model to infer procedural structure directly from visual observations.

(b.1) Immediate next-step prediction. Given a recent observation window and the task instruction, the model predicts the immediate next atomic action:

```
Given the recent observation and the task "{task}", predict the
immediate next atomic action the robot should execute.  Use a single
explicit verb-level description.
```

The target response is a single executable verb-level action.

(b.2) Future planning. Given a recent observation window and the task instruction, the model predicts the remaining atomic actions required to complete the task:

```
Given the recent observation and the task "{task}", list the
remaining atomic actions required to complete the task, starting
from the next time step.  Each action should be a single explicit
verb-level step.
```

The target response is an ordered list of remaining subtasks.

(c) Subtask-structured progress prediction. For progress prediction, the model first infers the remaining atomic actions and then estimates the current task completion percentage:

```
Given the recent observation and the task "{task}", first infer
the remaining atomic actions required to complete the task.  Then
estimate the current completion percentage and output it as a float
wrapped by <progress> tags.
```

The response ends with a structured progress tag:

$$\texttt{<progress>}\ p\ \texttt{\%</progress>},$$

where p is a continuous completion percentage. This format provides both token-level supervision for procedural reasoning and scalar supervision for the progress value head.

C.1.3 Training Sample Statistics

ProcVQA is split into a large-scale pretraining set and a curated refinement set. The first-stage set is constructed from the full annotated corpus and provides broad coverage across robots, environments, viewpoints, and task types. The second-stage set is built from human-selected trajectories with more accurate subtask alignment and clearer procedural structure.

Table 7: Summary of ProcVQA training data used for the two-stage ProcVLM training pipeline.

Statistic	Stage 1 Pretraining	Stage 2 Refinement
Data source	Full ProcVQA set	Human-selected subset
Token scale	$\sim$20B	2.8B
Trajectory scale	$\sim$400K	$\sim$15K
Main purpose	Broad procedural pretraining	High-quality refinement

To further characterize the procedural structure of the curated refinement set, we report the distribution of subtask counts per trajectory in Table 8. The Stage 2 set contains 13,688 trajectories, with an average of 2.87 subtasks per trajectory and a median of 3. This indicates that the curated set contains substantial multi-step procedural structure rather than only single-action executions, supporting its use for refinement-stage training on subtask alignment and progress reasoning.

C.2 ProcVLM Configuration

C.2.1 Backbone and Output Branches

ProcVLM is initialized from Qwen3-VL-2B-Instruct. The backbone takes visual observations and task instructions as multimodal input and generates textual responses through the standard autoregressive language modeling head. This language branch is used for all ProcVQA task families.

Table 8: Distribution of subtask counts per trajectory in the Stage 2 curated refinement set.

# Subtasks	1	2	3	4	5	6	7	8	9	10	11	12	≥ 14
# Trajectories	1455	5303	4187	1523	210	479	123	197	19	153	2	15	22

Summary statistic	Value
Total trajectories	13,688
Mean # subtasks	2.87
Median # subtasks	3
Minimum # subtasks	1
Maximum # subtasks	24

The value head is implemented as a three-layer MLP with hidden dimensions $d_h \to 4d_h \to d_h \to 1$, where d_h denotes the hidden size of the VLM backbone.

C.2.2 Progress-Value Gating

The progress value head is controlled by a semantic gating mechanism. During training, this branch is activated when the ground-truth response contains a `<progress>` tag. For samples without progress supervision, only the language modeling objective is applied. During inference, the value branch can be activated on demand for responses containing a `<progress>` tag. This gated design separates general procedure-aware text generation from continuous value prediction. It allows ProcVLM to learn from all task families through language modeling, while using progress-prediction samples to additionally supervise the value head.

C.2.3 Value-Head Pooling

Let $H = \{h_i\}_{i=1}^{L}$ denote the sequence hidden states produced by the VLM backbone. For progress-prediction samples, the value head applies attention pooling over the valid sequence positions:

$$\alpha_i = \frac{\exp(s(\tilde{h}_i)) \cdot m_i}{\sum_{j=1}^{L} \exp(s(\tilde{h}_j)) \cdot m_j},$$

$$h_{\text{pool}} = \sum_{i=1}^{L} \alpha_i \tilde{h}_i, \qquad \hat{p} = f_{\text{value}}(h_{\text{pool}}),$$

where $s(\cdot)$ is the attention scoring function, f_{value} is the regression head, and m_i denotes the attention mask that excludes padding tokens. During training, $\tilde{h}_i$ is obtained by applying feature-level dropout to the tail hidden states that contain the progress answer, while keeping other hidden states unchanged. This corruption reduces direct access to the ground-truth progress tokens during teacher forcing and encourages the value head to infer progress from the visual observation, task instruction, and prior procedural reasoning.

C.3 Training Pipeline and Implementation Details

C.3.1 Language Modeling Supervision

Given the ProcVQA templates above, all task families are trained with the standard autoregressive language modeling objective over the supervised assistant response. The input prompt, including task instructions and visual observations, is used as conditioning context, while the loss is applied only to the target response tokens. Formally, given a multimodal input x and a target response $y = \{y_t\}_{t=1}^{T}$, we compute

$$\mathcal{L}_{\text{LM}} = -\frac{1}{|\mathcal{M}_{\text{text}}|} \sum_{t \in \mathcal{M}_{\text{text}}} \log P_\theta(y_t \mid y_{<t}, x),$$

where $\mathcal{M}_{\text{text}}$ denotes the set of target response tokens used for language modeling supervision. Prompt tokens and padding tokens are masked out in the label tensor and are therefore excluded from the loss.

Progress-prediction samples are also included in this language modeling objective. In these samples, the reasoning context and the formatted progress answer are learned as text, while the scalar value inside the `<progress>` tag is additionally used as supervision for the value head.

C.3.2 Progress Value Supervision

For progress-prediction samples, we parse the scalar completion value from the structured `<progress>` tag and use it as the ground-truth progress label. Although the textual response expresses progress as a percentage, the regression loss is computed on a normalized scale. Let p denote the parsed completion percentage and $\bar{p} = p/100$ denote its normalized value. The value head predicts a normalized progress score $\hat{p} \in [0, 1]$, and the value loss is defined as

$$\mathcal{L}_{\text{value}} = \ell_{\text{reg}}(\hat{p}, \bar{p}),$$

where ℓ_{reg} is the regression loss used for continuous progress prediction.

The overall objective is

$$\mathcal{L} = \mathcal{L}_{\text{LM}} + \lambda \cdot \mathbb{I}_{\text{prog}} \mathcal{L}_{\text{value}},$$

where $\mathbb{I}_{\text{prog}}$ indicates whether the sample contains progress supervision and λ controls the contribution of the value regression loss. In minibatches containing both progress and non-progress samples, the value loss is averaged only over samples with valid progress labels.

C.3.3 Leakage-Prevention Tail Dropout

During teacher forcing, the ground-truth progress value appears near the end of the textual target. If the value head directly relies on the hidden states of the `<progress>` answer tokens, it may recover the target value from the answer span rather than infer task progress from the visual observation and procedural context. To reduce this leakage, we apply a soft tail-masking strategy before value-head pooling.

Specifically, for each training sequence, we identify the last valid token according to the attention mask and apply feature-level dropout to the final N_{tail} hidden states:

$$\tilde{h}_i = \begin{cases} \text{Dropout}(h_i; p_{\text{tail}}), & i \in \mathcal{T}_{\text{tail}}, \\ h_i, & \text{otherwise}, \end{cases}$$

where $\mathcal{T}_{\text{tail}}$ denotes the tail token region containing the progress answer. In our implementation, we set $N_{\text{tail}} = 14$, which covers the numerical progress answer, the surrounding `<progress>` tags, and the sequence ending tokens in the default response format. We use $p_{\text{tail}} = 0.5$ for feature-level dropout during training.

The corrupted hidden states $\tilde{H} = \{\tilde{h}_i\}_{i=1}^{L}$ are then passed to the attention pooler together with the standard attention mask. Importantly, the tail tokens are not removed from pooling by setting their attention mask to zero. Instead, their hidden features are partially corrupted. This soft masking weakens direct access to the ground-truth progress tokens while keeping the tail positions in the computation graph, allowing gradients to still flow through the complete response sequence.

At inference time, no tail dropout is applied. When the value branch is enabled, the model runs a forward pass over the generated sequence and predicts the progress value from the resulting hidden states.

C.3.4 Two-stage Training Pipeline

As described in Section 3.2.3, ProcVLM is trained in two stages: large-scale pretraining on the full ProcVQA dataset, followed by refinement on a curated subset constructed from human-selected trajectories. We provide the training curves in Figure 5 to analyze the distinct roles of the two stages.

Figure 5 reports representative metrics throughout training on both the in-distribution `test_full` split and the out-of-distribution `valid_full` split. F1@50 directly measures action segmentation quality by matching predicted and ground-truth subtask intervals. SA evaluates the semantic accuracy of predicted subtask plans. GS measures progress regression accuracy. MCC@95 evaluates success-state prediction by converting progress into a binary decision, where $p \geq 95$ and $\hat{p} \geq 95$ indicate ground-truth and predicted success, respectively. VOC and KT both measure intra-trajectory ordering ability, i.e., whether predicted progress values preserve the temporal order of states within a trajectory.

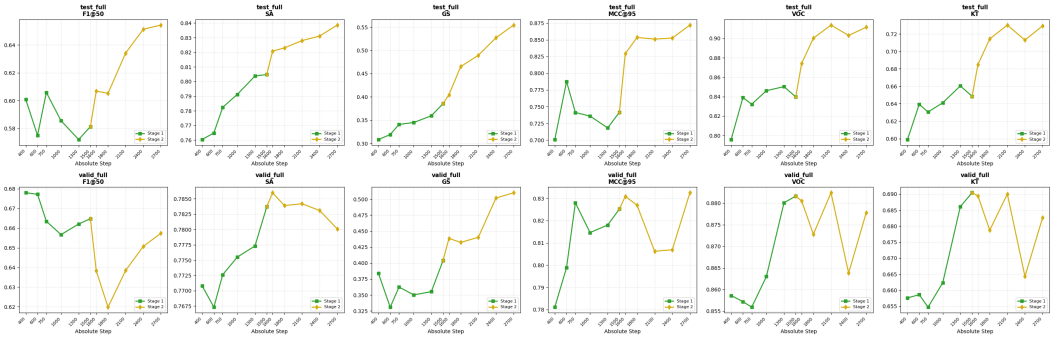


Figure 5: Training curves of ProcVLM under the two-stage training pipeline. Green denotes Stage 1 pretraining on the full ProcVQA dataset, and yellow denotes Stage 2 refinement on the curated subset. Metrics are evaluated on the in-distribution `test_full` split and the out-of-distribution `valid_full` split. Stage 1 mainly establishes generalizable procedural representations from large-scale diverse supervision, while Stage 2 further improves prediction precision with higher-quality refinement data.

The Stage 1 curves show that large-scale pretraining is crucial for generalization. Training on the full ProcVQA dataset consistently improves planning, progress estimation, success prediction, and trajectory-ordering metrics on both in-distribution and out-of-distribution splits, indicating that diverse procedure-aware supervision helps learn transferable task-progress representations.

Stage 2 further refines the pretrained model on the curated subset. After switching to higher-quality data, the model gains precision on metrics requiring accurate subtask boundaries, calibrated progress values, and reliable temporal ordering, with stronger improvements on the in-distribution split and smaller gains or fluctuations on the out-of-distribution split. Together, these trends support the two-stage design: Stage 1 builds generalizable procedural representations, while Stage 2 sharpens subtask and progress prediction.

C.3.5 Training Hyperparameters

The main training hyperparameters are summarized in Table 9. Both stages use the same joint objective described above and train the VLM backbone together with the progress value head. They differ mainly in data distribution, optimization schedule, and value-head regularization: Stage 1 uses a larger learning rate and lower value-head regularization for broad pretraining, while Stage 2 uses a smaller learning rate and stronger refinement regularization on the curated subset.

D ProcVQA Evaluation Details

This section provides additional details of the ProcVQA benchmark and evaluation metrics used in Section 4.1.

D.1 Benchmark and Splits

ProcVQA is a human-selected ECoT-based VQA benchmark for embodied procedural understanding. It covers three manipulation-understanding tasks: action segmentation, future planning, and task progress estimation. To evaluate both in-domain performance and cross-domain generalization, we divide ProcVQA into an in-distribution (ID) split and an out-of-distribution (OOD) split. The ID split is derived from training-domain datasets, including DROID, Bridge, Table30, and selected OXE subsets, while the OOD split is constructed from unseen RoboTwin tasks.

D.2 Metrics

Boundary F1 score (BF1). BF1 is calculated as follows. Given predicted and ground-truth action segments, we first extract segment boundaries and remove duplicated boundary positions. A predicted boundary is matched to a ground-truth boundary if their temporal distance is within 5% of the sequence length. Each boundary can be matched at most once. BF1@5 is then computed as the

Table 9: Training hyperparameters for the two-stage ProcVLM training pipeline.

Hyperparameter	Stage 1 Pretraining	Stage 2 Refinement
Training data	`train_s1_8k`	`train_s2_8k`
Token scale	$\sim$20B	2.8B
Backbone initialization	Qwen3-VL-2B-Instruct	Stage 1 checkpoint
Trainable modules	VLM backbone, vision tower, multimodal projector, value head	VLM backbone, vision tower, multimodal projector, value head
LoRA	Disabled	Disabled
Precision	bf16	bf16
Context length	8192	8192
Data packing	Enabled	Enabled
Per-device batch size	1	1
Gradient accumulation	64	64
Effective batch size	$64 \times N_{\mathrm{GPU}}$	$64 \times N_{\mathrm{GPU}}$
Base learning rate	1×10^{-5}	5×10^{-6}
Multimodal projector learning rate	1×10^{-5}	4×10^{-6}
Vision tower learning rate	5×10^{-6}	2×10^{-6}
Optimizer	AdamW	AdamW
Learning-rate schedule	Cosine	Cosine
Warmup ratio	0.03	0.05
Weight decay	0.01	0.01
Training length	$\sim$1.1–1.2 epochs	$\sim$3–4 epochs
Image pixel range	32^2–512^2	32^2–512^2
Video sampling	2 FPS, up to 512 frames	2 FPS, up to 512 frames
Regression loss ℓ_{reg}	L_1	L_1
Value-loss weight λ	0.2	0.3
Value-head dropout	0	0.05
Value noise std.	0.03	0.01
Distributed training	DeepSpeed ZeRO-2	DeepSpeed ZeRO-2 with offload
Gradient checkpointing	Enabled	Enabled

boundary-level F1 score from matched and unmatched boundaries. We use BF1@5 as the primary segmentation metric because it penalizes both missing and redundant boundaries.

Matched boundary localization error (mMAE). mMAE measures the temporal localization error of matched boundaries. It is computed as the mean absolute distance between each matched predicted boundary and its corresponding ground-truth boundary. Since mMAE is calculated only over matched boundaries, it does not penalize false positives or false negatives. We therefore report it as an auxiliary metric rather than the primary action segmentation metric.

Future planning success (Success). Success measures whether the model-generated future task plan can be successfully executed and satisfy the given instruction. We compute this metric through anonymized and randomly ordered human evaluation, where annotators compare model outputs without knowing the model identity.

Value-Order Correlation (VOC). VOC evaluates the ranking accuracy of predicted task progress within each trajectory. We compute VOC using Spearman's rank correlation coefficient ρ between the predicted progress values and the ground-truth temporal progress labels. This metric focuses on whether the model preserves the correct progress order along a trajectory.

Effective Progress Resolution (EPR). EPR measures the density of continuous progress regression. It discourages models from obtaining high VOC by predicting only a few discrete progress anchors, such as 0.25, 0.5, 0.75, and 1.0. For predicted progress values $\hat{p}$, EPR is defined as

$$\mathrm{EPR}_\tau(\hat{p}) = -\log_2 \min\left\{\Delta \in \{1/k \mid k \in \mathbb{N}_+\} \mid \Delta \cdot |\mathcal{B}_\Delta(\hat{p})| \geq \tau\right\},$$

where $\mathcal{B}_\Delta(\hat{p})$ denotes the set of occupied quantization bins after quantizing $\hat{p}$ with bin width Δ. In our experiments, we report EPR@50 by setting $\tau = 0.5$.

E Reward Model Baseline Adaptation Details

This section provides additional details of the reward model baseline adaptation used in Section 4.2.1.

Evaluation setting. We evaluate ProcVLM against generalizable manipulation reward models with visual-language inputs. We focus on models that can be adapted to estimate absolute task progress from short local temporal windows, typically 1–16 consecutive frames within about one second. We select RoboDopamine and Robometer as representative baselines for this setting.

RoboDopamine. RoboDopamine trains a step-aware vision-language reward model to estimate fine-grained manipulation progress from multi-view state observations by predicting relative progress hops and fusing them into dense rewards. Since RoboDopamine requires contrastive inputs, we concatenate query frames from the same trajectory in the original ProcVQA order and prepend a blank image as a neutral contrastive start anchor. This construction avoids giving the model additional visual progress cues beyond the queried observations. Unless otherwise specified, RoboDopamine results are obtained with the Robo-Dopamine-GRM-2.0 series, and we use the corresponding 4B or 8B variant according to each experiment.

Robometer. Robometer trains a Qwen3VL-based video-language reward model with both frame-level progress/success supervision and trajectory-comparison preference supervision, enabling dense reward prediction from robot videos and language instructions. In our evaluation, Robometer receives the instantaneous input window from each VQA query, and we use its last-frame prediction as the final progress estimate.

F RoboFAC Evaluation Details

This section provides additional details of the RoboFAC evaluation protocol and metric definitions used in Section 4.2.2.

F.1 Implementation Details

Benchmark. RoboFAC is a failure-centric robotic manipulation VQA benchmark for robotic failure analysis and correction. It contains diverse successful and failed manipulation executions, with structured annotations for task understanding, failure analysis, and correction. RoboFAC consists of both real-robot and simulated data, and we use only the real-robot subset for evaluation. We use RoboFAC to evaluate whether ProcVLM can generalize to unseen tasks with only a small number of demonstrations.

One-shot split construction. We first split all trajectories into success and failure sets. We then construct task-level one-shot training sets separately from these two sets. For the success set, we select one trajectory for each task as the one-shot successful demonstration. For the failure set, we select one trajectory for each task–failure-type pair, so that different failure modes are covered with minimal supervision. All remaining trajectories are used for testing.

Viewpoint coverage. The one-shot demonstrations are selected at the trajectory level and do not explicitly cover all camera viewpoints. As a result, the test set may contain observations from viewpoints that are absent from the one-shot demonstrations. This design makes the evaluation more challenging and better reflects practical reward-model deployment, where a small number of demonstrations cannot exhaustively cover all visual variations.

Success detection. For binary success detection, each model performs inference on the full test set using the visual observation from the last temporal window of each trajectory. Robometer predicts completion with its success head, whose probability is discretized into a binary success label. For RoboDopamine and ProcVLM, we use the predicted progress value and apply a fixed success threshold to obtain the binary completion label.

Full-trajectory progress evaluation. For full-trajectory progress evaluation, running all test trajectories is computationally expensive. We therefore sample 100 test trajectories in total, evenly from the success and failure sets, and evaluate progress prediction along the sampled trajectories.

F.2 Metrics

VOC on successful trajectories. $\mathrm{VOC}_{\mathrm{succ}}$ is computed only on successful trajectories and follows the same definition as VOC in ProcVQA. It measures whether predicted progress values preserve the correct temporal order along a successful execution.

MAE on failed trajectories. $\mathrm{MAE}_{\mathrm{fail}}$ is computed only on failed trajectories. For each failed trajectory, we identify the earliest frame that reaches the maximum predicted progress value:

$$t^* = \min \arg\max_t \hat{p}_t,$$

where $\hat{p}_t$ denotes the predicted progress value at frame t. We treat t^* as the predicted turning point, corresponding to the estimated failure onset. Given the human-annotated cutoff index t_{cut}, $\mathrm{MAE}_{\mathrm{fail}}$ is computed as

$$\mathrm{MAE}_{\mathrm{fail}} = \frac{1}{N_{\mathrm{fail}}} \sum_{i=1}^{N_{\mathrm{fail}}} \left| t^{*(i)} - t_{\mathrm{cut}}^{(i)} \right|.$$

MCC for success detection. We use the Matthews correlation coefficient (MCC) to evaluate binary success detection. Predicted progress scores or success probabilities are converted into completion labels using a fixed threshold, and MCC is then computed against the ground-truth success/failure labels.

Latency. Latency is measured in seconds during full-trajectory progress evaluation and reports the average inference time required to process one trajectory. We measure latency on NVIDIA RTX A6000 GPUs, using single-GPU deployment for all models except Qwen3.5-27B, which is deployed with 4-way tensor parallelism.

G Real-Robot Experiment Setup

This section provides additional details on real robot reward finetuning experiments in Section 4.3.

Task and data setup. We conduct real-robot experiments on a tabletop stack-bowls task with a single-arm JAKA robot. The task requires the robot to pick up a bowl on the table and place it into a target bowl, forming a properly nested bowl stack. The real-robot training set contains 50 human teleoperated demonstration trajectories, approximately evenly distributed across scenes with two, three, and four bowls. We train both SFT and RFT policies for 10k steps with a batch size of 64, and visualize representative rollouts at both 5k and 10k training steps. Due to the higher difficulty of the three-bowl and four-bowl settings and our limited real-robot evaluation budget, preliminary runs showed very low success rates on these settings. Therefore, we restrict the real-robot evaluation to the two-bowl setting, where each rollout requires one bowl to be placed into the target bowl.

Evaluation protocol. For each method, we conduct 12 real-robot rollouts under the two-bowl setting. Success is measured by the degree of proper nesting in the final bowl configuration. For each rollout, we assign a nesting score $s \in [0, 1]$ according to the following rubric:

- $s = 0.0$ **(failure)**: the stacking action is not completed within 30 seconds, such as when the bowl is not grasped or the robot does not attempt insertion.
- $s = 0.5$ **(partial success)**: the robot attempts to insert the bowl, but the placed bowl slips out of the target bowl or is placed immediately next to the target bowl.
- $s = 1.0$ **(full success)**: the robot successfully places the bowl inside the target bowl.

The final score of each method is computed as the average nesting score over 12 rollouts. We report this value as the soft success rate, where partial placements receive half credit.

Rollout visualization. Figure 6 shows representative real-robot rollout records under the two-bowl evaluation setting. These examples include different execution outcomes and illustrate how the final nesting score is assigned based on the resulting bowl configuration.

Table 10: Real-robot experimental setup on the JAKA stack-bowls task.

Item	Description
Robot platform	JAKA robot, single arm
Scene	Tabletop stack-bowls task
Training data	50 human teleoperated demonstrations
Demonstration settings	Two-bowl, three-bowl, and four-bowl scenes
Evaluation setting	Two-bowl scene only
Rollouts per method	12 rollouts
Metric	Soft success rate

H Reward Modeling Cases

Zero-shot cases. We first evaluate ProcVLM in a fully zero-shot setting on multiple tasks from RoboMIND and RoboTwin. As shown in Figure 7, ProcVLM can identify the current action stage, reason about remaining steps, and produce task-conditioned progress feedback without task-specific demonstrations or reward-model adaptation. These cases illustrate the transferable progress perception learned from procedure-aware pretraining.

Zero-shot reward editing. We finally present a zero-shot reward editing case on a RoboMIND pick-and-place task. As shown in Figure 8, ProcVLM is applied to the same video under two task instructions: "put the apple into the basket" and "put the apple into the basket and move the basket to the upper corner." The resulting reward curves change with the edited task description, showing that ProcVLM grounds reward prediction in task-conditioned subtask structure rather than only superficial visual progress.

One-shot adaptation cases. We further evaluate ProcVLM under a one-shot adaptation setting, where only one successful demonstration is used for LoRA tuning in each scenario. As shown in Figures 9, 10, 11, and 12, ProcVLM quickly adapts to RoboMIND and RoboFAC scenarios containing successful, failed, and retry executions, providing meaningful progress feedback beyond the demonstrated success trajectory.



SFT, 5k training steps, grasp failed.



SFT, 5k training steps, partial success.



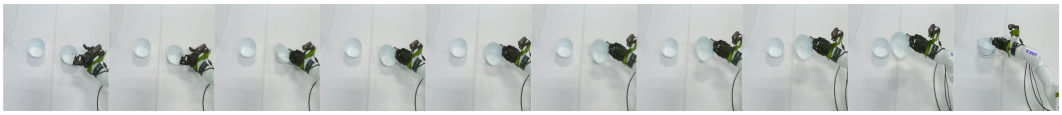
RFT, 5k training steps, partial success.



RFT, 5k training steps, full success.



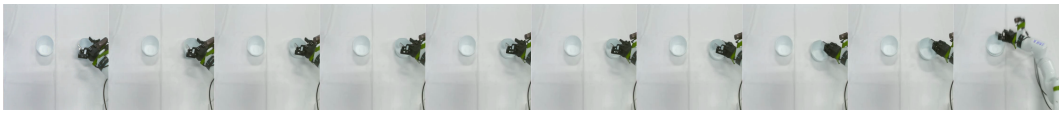
SFT, 10k training steps, partial success.



SFT, 10k training steps, full success.



RFT, 10k training steps, partial success.



RFT, 10k training steps, full success.

Figure 6: Representative real-robot rollout records on the JAKA tabletop stack-bowls task. We show examples from SFT and RFT policies at both 5k and 10k training steps. Each method is evaluated with 12 rollouts under the two-bowl setting, and the final score is computed from the nesting quality of the placed bowl.

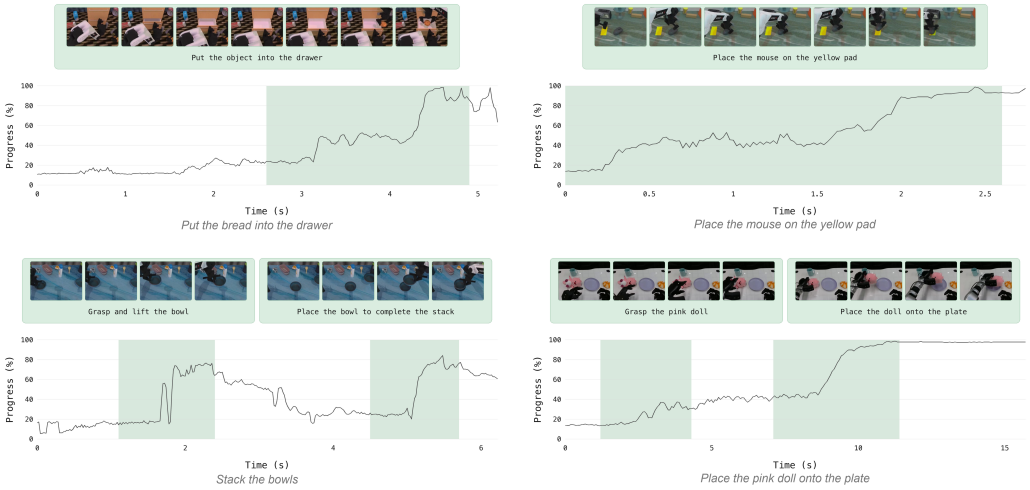


Figure 7: Zero-shot reward modeling cases on RoboMIND and RoboTwin. ProcVLM provides task-conditioned progress feedback across different manipulation tasks without task-specific adaptation.

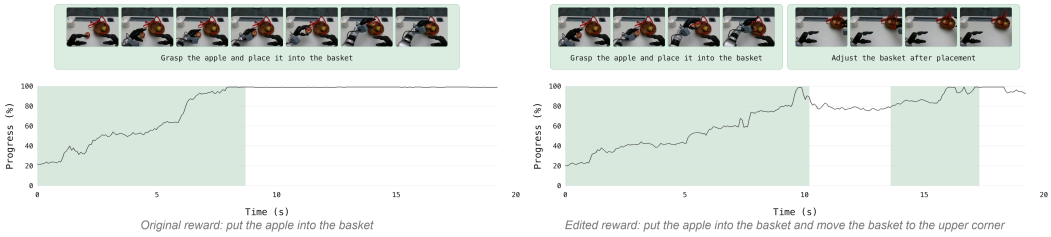


Figure 8: Zero-shot reward editing on the same video sequence. Left: reward for "put the apple into the basket." Right: edited reward for "put the apple into the basket and move the basket to the upper corner."

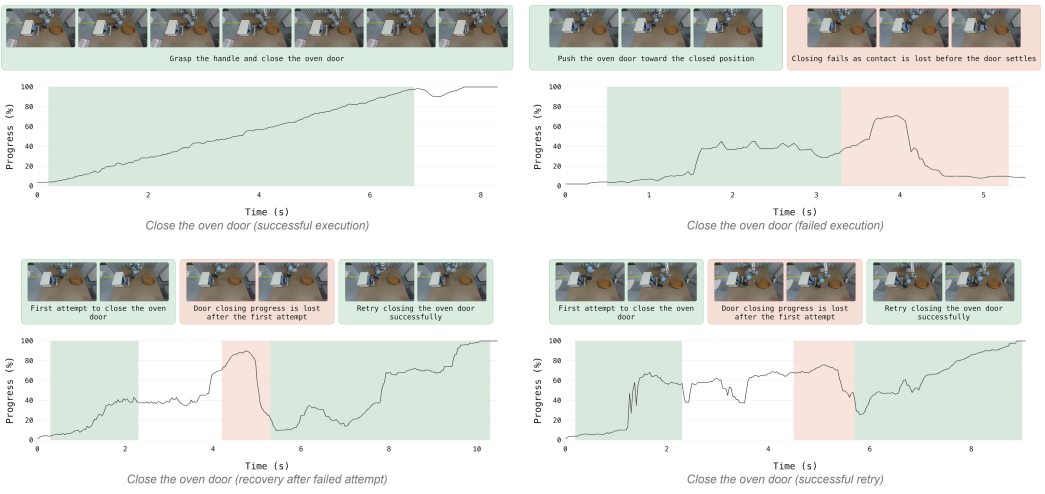


Figure 9: One-shot adaptation case for a close-oven task. ProcVLM adapts from one successful demonstration and evaluates different execution outcomes, including failures and retry-based recoveries.

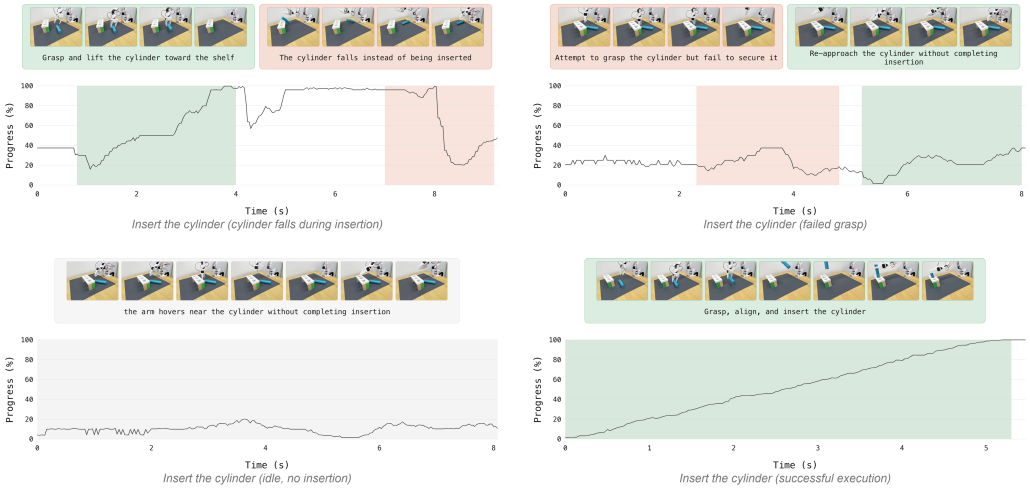


Figure 10: One-shot adaptation case for an insert-cylinder task. ProcVLM transfers the demonstrated task structure to executions with successful, failed, and idle behaviors.

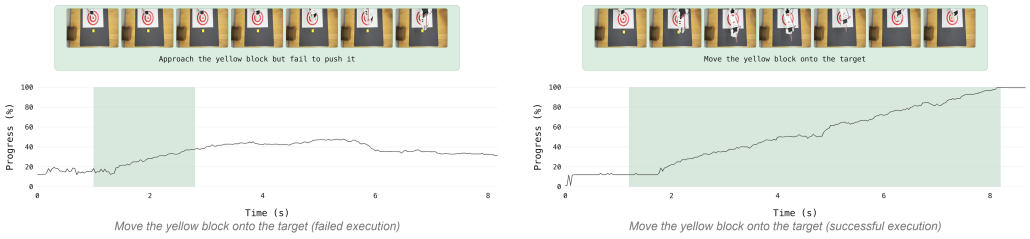


Figure 11: One-shot adaptation case for a move-target task. ProcVLM transfers the demonstrated task structure to successful and failed executions.

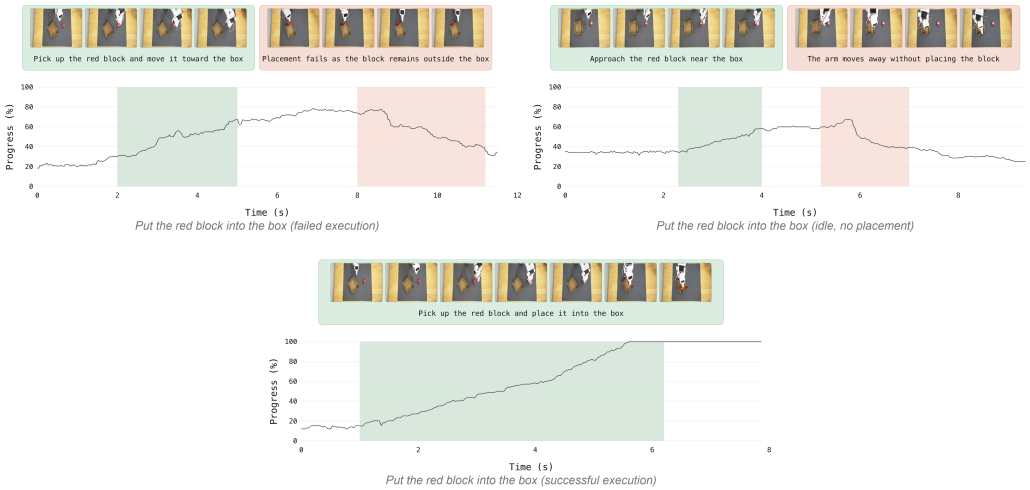


Figure 12: One-shot adaptation case for a put-in-box task. ProcVLM generalizes from one successful demonstration to evaluate successful and failed executions.